

课程笔记

CS336 第 17 讲：对齐—多模态

课程笔记整理

2026 年 6 月 9 日



视频作者/频道：Stanford CS336（中配 + 中字）

发布日期：2026 年 4 月 15 日

视频时长：约 77 分钟（01:17:39）

视频链接：<https://www.bilibili.com/video/BV12gLx6xEcq?p=17>

目录

1 引言与多模态概述	4
1.1 为何语言模型课程需要多模态	4
1.2 全模态的理想	4
1.3 Token 概念的扩展	4
1.4 本讲聚焦的核心问题	5
1.5 本章小结	6
2 总结与前沿反思	6
2.1 讲者的实质性收尾讨论	6
2.1.1 ”原生多模态”的宣传与现实	6
2.1.2 理解与生成的不对称性	7
2.1.3 模态信息密度的差异	7
2.2 结构化提炼：核心主张与机制	7
2.3 跨章节联系与概念压缩	8
2.4 具体收获与开放问题	8
2.4.1 具体收获	8
2.4.2 开放问题	9
2.5 下一步	9
3 CLIP：对比语言-图像预训练	10
3.1 历史背景与动机	10
3.2 核心对比学习机制	10
3.3 数据构建	12
3.4 图像预处理	12
3.5 视觉编码器	12
3.5.1 ViT 图像分块与位置嵌入	13
3.5.2 Attention Pooling	13
3.6 文本编码器	14
3.7 零样本分类	14
3.8 噪声鲁棒性分析	15
3.9 对比实验：预测文本 vs 对比学习	15
3.10 CLIP 的局限性	16
3.11 本章小结	17
4 SigLIP：解耦批大小的对比学习	17
4.1 从 Softmax 到 Sigmoid：损失函数的重新设计	18
4.2 无需复杂负采样策略的简洁性	19
4.3 分布式训练中的环状通信	19
4.4 训练效率对比：CLIP vs SigLIP	20
4.5 本章小结	21

5 视觉语言模型架构：LLaVA 与 Qwen-VL	21
5.1 VLM 的通用架构范式	21
5.2 LLaVA：两阶段训练与合成数据	22
5.2.1 模型组件	22
5.2.2 两阶段训练策略	22
5.2.3 合成数据生成	23
5.3 LLaVA 1.5 的改进：AnyRes 与多模态扩展	23
5.3.1 AnyRes：任意分辨率处理	23
5.3.2 多图像与视频支持	24
5.3.3 模态自适应的数据处理	24
5.3.4 三阶段训练与跨模态迁移	25
5.3.5 视觉提示扩展	25
5.4 Qwen-VL：交叉注意力与 2D 位置感知	25
5.4.1 模型架构	25
5.4.2 特殊 Token 设计	26
5.4.3 三阶段训练	26
5.5 LLaVA 与 Qwen-VL 的对比	27
5.6 本章小结	27
6 Qwen2-VL 与 Qwen3-VL 的演进	28
6.1 Qwen2-VL：动态分辨率与 MRoPE	28
6.1.1 更大的视觉编码器与动态分辨率	28
6.1.2 视频处理：2fps 采样与 16K 上限	29
6.1.3 MRoPE：将 RoPE 扩展到三维空间	29
6.1.4 显式视频时间戳 token	30
6.1.5 长度平方根归一化	30
6.2 Qwen3-VL：深度嵌入与上下文扩展	31
6.2.1 基于 Qwen3 的语言模型底座	31
6.2.2 上下文长度的逐步扩展：8K $\rightarrow$ 32K $\rightarrow$ 256K	31
6.2.3 视觉编码器升级：Clip2	32
6.2.4 3D RoPE 的维度交错改进	32
6.2.5 深度嵌入适配器：从黑箱投影到残差流注入	32
6.2.6 后训练：SFT + 知识蒸馏 + RL	33
6.2.7 基准表现	33
6.3 本章小结	34
7 Chameleon：全离散化统一多模态	35
7.1 从连续编码到离散统一	35
7.2 VQ-VAE：图像的离散化编码	35
7.3 训练：回归普通语言模型	36
7.4 实际挑战与局限性	37
7.4.1 训练不稳定性	37
7.4.2 离散化的信息丢失	37

7.4.3	多模态联合训练的困难	38
7.5	历史地位：从流行到式微	38
7.6	本章小结	39

1 引言与多模态概述

1.1 为何语言模型课程需要多模态

在本课程的前十六讲中，我们始终围绕语言模型展开讨论。从 Transformer 架构到预训练、微调、对齐，再到推理与系统优化，语言模型已经展现出惊人的通用性——它们能够将任意文本映射为任意文本，无论是跨语言翻译、代码生成、诗歌创作，还是 DNA 序列分析，皆可胜任。然而，正如讲者在讲座开篇所指出的那样，现实世界是多模态的：我们日常接触的信息不仅包含文本，还包括图像、音频、视频，以及它们之间复杂的交织关系。如果一门关于现代人工智能的课程不涉及多模态内容，便显得不够完整。

核心观察

当前所有主要的大规模模型（如 GPT-4o、Gemini 系列）都已具备多模态能力。多模态不再是边缘方向，而是基础模型的标准配置。

1.2 全模态的理想

讲者在这一部分提出了一个雄心勃勃的理想目标——**全模态**（Omnimodal）能力。所谓全模态，是指模型能够处理任意模态组合的输入，并生成任意模态的输出。具体而言：

- **多模态输入**：同时接收图像、视频、音频与文本的组合；
- **跨模态问答**：针对一张图片和一段视频提出问题，模型能够理解并回答；
- **多模态生成**：不仅能生成文本，还能生成图像，甚至将音频转换为图像；
- **模态交织**：文本与图像交替出现，形成连贯的多模态对话。

从专用模型到基础模型的范式转移

回顾人工智能的发展历程，我们见证了从专用模型（如仅做图像分类的 ResNet、仅做机器翻译的 Seq2Seq）向基础模型（Foundation Model）的范式转移。语言模型率先完成了这一跃迁——一个模型通过大规模预训练即可应对多种文本任务。多模态领域正在经历同样的过程：从为每个模态组合单独设计模型，走向一个统一的全模态基础模型。

讲者同时坦诚地指出，本讲不会构建完整的全模态模型，而是聚焦于理解图像的核心组件——即如何将非文本模态有效地输入到 Transformer 中。这一聚焦既符合课程时间限制，也反映了当前技术的主流路径：在输入侧实现多模态理解，而生成侧仍主要依赖文本输出（图像生成则交由扩散模型等专门模块处理）。

1.3 Token 概念的扩展

Transformer 最初为处理文本而设计，其核心抽象是 **token**——离散的语义单元。在文本领域，token 通过 BPE（Byte Pair Encoding）等分词算法获得，每个 token 对应一个子词片段，具有一定的语义意义。然而，当我们将 Transformer 扩展到图像、音频、视频等模态时，必须重新审视 token 的定义。

讲者提出了一个关键的扩展视角：

Token 的广义定义

Token 不仅可以是离散的文本符号，也可以是**连续的嵌入向量** (continuous embedding)。广义而言，一个 token 代表一个**语义信息单元** (semantic information unit)。

这一扩展具有重要的理论意义。在自然语言中，子词 token（如“playing”分解为“play”+“ing”）携带可解释的语义；相比之下，单个像素本身几乎不携带任何语义信息——一张蓝色像素既可能是天空，也可能是海洋，或是某件衣物。因此，对于非文本模态，我们需要找到一种“相当于 BPE 分词器”的机制，将原始感知数据转化为 Transformer 可理解的语义单元。

常见误解：像素即 token

初学者容易误认为可以直接将图像的每个像素作为 token 输入 Transformer。这种做法的问题在于：

1. 单个像素缺乏语义，与文本 token 的语义承载能力不对等；
2. 高分辨率图像会产生海量像素（如 1024×1024 图像有超过 100 万个像素），计算代价不可接受；
3. 像素间的空间关系需要额外的位置编码机制来捕捉。

因此，我们需要更高级别的表示方法——这正是后续章节中 CLIP、VQ-VAE 等方法的核心价值。

1.4 本讲聚焦的核心问题

基于上述分析，讲者提炼出两个根本性问题：

1. **输入问题**：如何将非文本模态（图像、视频、音频）转化为 Transformer 可以处理的 token 形式（离散或连续）？
2. **生成问题**：如何让 Transformer 生成非文本模态的输出？

本讲主要聚焦于第一个问题——输入侧的多模态对齐。这是因为在当前技术路径中，理解（将图像转化为语言模型可处理的表示）是生成的前提，且已有相对成熟的解决方案；而真正的原生多模态生成（由同一个模型直接输出图像 token）仍面临训练稳定性、信息保真度等重大挑战。

文本分词 vs. 图像分词

即使是文本分词，也并非完美无缺。讲者提到，人们或许希望拥有更好的分词器，甚至完全不需要分词器（如字节级模型）。但文本分词的困难程度远不及图像、音频等非文本模态——BPE 分词器至少能将文本可靠地分解为可管理的离散单元。而对于图像，我们需要回答：什么是一个“图像子词”？一个物体？一个区域？一种纹理？这些问题至今没有唯一答案，不同的设计选择催生了 CLIP 的连续嵌入、VQ-VAE 的离散码本、以及扩散模型的渐进去噪等截然不同的技术路线。

1.5 本章小结

本章从语言模型的通用性出发，指出现实世界的多模态本质，进而提出了**全模态** (Omnimodal) 的理想目标。核心论点可以总结为：

- Transformer 已成为跨模态处理的最佳架构选择，但其原生设计面向文本 token；
- 必须将 token 概念从离散符号扩展为连续语义单元，才能容纳图像、音频、视频等模态；
- 本讲的核心任务是探索如何将非文本模态”分词”为 Transformer 可理解的形式。

带着这些问题，我们自然地将目光投向 2021 年——一个里程碑式的工作即将登场。OpenAI 提出的 CLIP 模型首次展示了如何通过对比学习，将图像和文本映射到统一的嵌入空间，为后续所有视觉语言模型奠定了基石。下一章将深入剖析 CLIP 的设计动机、核心机制与历史意义。

2 总结与前沿反思

2.1 讲者的实质性收尾讨论

在讲座的最后几分钟，讲者从前述具体技术细节中抽离出来，对多模态领域的前沿现状进行了高屋建瓴的反思。这些反思并非简单的内容复述，而是基于整讲技术脉络的深层判断。

2.1.1 ”原生多模态”的宣传与现实

讲者首先对当前前沿模型（如 Gemini、GPT-4o）所宣传的”原生多模态” (native multimodality) 能力进行了审慎的解读。这些模型确实能够处理多种模态的输入与输出，但其内部架构的具体细节并未公开。讲者推测，为了避免信息丢失，这些系统很可能在底层结合了**连续编码技术** (continuous encoding)，而非完全依赖离散 token 化方案。对于图像生成部分，**扩散模型** (Diffusion Model) 很可能仍扮演着关键角色。

281

282

283

- VQ-VAE (Vector Quantized Variational Autoencoder) [Oord+ 2017]
- Idea: map image to a discrete codebook, decode back

Input



Encoder

$q_\phi(z|x)$

Latent features (continuous)



Vector Quantization

Nearest code



Quantized (latent indices)

3	12	7
4	4	1
9	18	2

Codebook (Embedding Table) $E \in \mathbb{R}^{K \times D}$

e_1		...	e_{K-1}	
e_2			e_K	

Loss

$\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{vq}}$

- $\mathcal{L}_{\text{recon}}$: reconstruction loss (e.g., L1 or L2)
- $\mathcal{L}_{\text{vq}}$: vector quantization loss (commitment to discrete codes)

Neural Discrete Representation Learning

Aaron van den Oord, Oriol Vinyals, Krzysztof Szepesvári

2017-11-02

Learning useful representations without supervised learning. In this paper, we propose a simple model of discrete representations. Our model, the VQ-VAE, differs from VAEs in two key ways: 1) it uses discrete codes; and the prior is learned from data. This allows the model to circumvent issues of when they are paired with a powerful autoencoder framework. Pairing these representations with a powerful autoencoder framework allows the model to generate high quality images, videos, and audio, and conversion and unsupervised learning of representations.

随后扩散模型逐渐流行，成为了生成任务中的有力工具

diffusion models came out and became popular and viable for generation.

284

285

- Encodes 512 x 512 image into 1024 tokens (codebook of size 8192)

图 1: 讲者对前沿多模态模型的技术反思：原生多模态宣传与现实的差距

2.1.2 理解与生成的不对称性

讲者提出了一个深刻且常被忽视的洞察：**理解** (understanding) 与 **生成** (generation) 之间存在根本性的不对称，不存在一种通用编码器能够同时最优地服务于所有任务。

以 CLIP 为例：它的设计目标是图像分类与语义对齐，因此只需要捕获**高层次语义** (high-level semantics)。一个紧凑的向量（如 768 维或 1024 维）足以表示“图中有一只狗”这样的抽象概念。然而，如果任务变为 OCR（光学字符识别）或图像生成，这种粗粒度的表示就远远不够了——我们需要知道每个字母的确切形状、位置、颜色，需要**细粒度细节** (fine-grained details)。

扩散模型之所以在生成任务上表现优异，正是因为它能够在从低频到高频的所有尺度上逐步优化信息，保留生成高质量图像所需的全部细节。这与 CLIP 那种“一锤定音”式的单向量表示形成了鲜明对比。

编码器粒度的权衡律

不同任务对编码器粒度有不同需求：

- **分类/检索** → 高层语义 → 小向量（如 CLIP 的嵌入）
- **OCR/生成** → 细粒度细节 → 扩散模型或高分辨率特征图

没有免费的午餐——一个编码器无法同时在两个极端上都达到最优。

2.1.3 模态信息密度的差异

讲者进一步指出，不同模态的信息密度存在显著差异，这直接影响训练策略的设计。具体而言，**视频的信息密度通常低于文本**——一段视频中的大量帧可能只包含有限的新信息（例如，一个静态场景中连续多帧几乎相同）。如果不加处理，视频样本会在训练损失中占据主导地位，稀释文本信号的影响。

这一观察直接解释了 Qwen2-VL 中**长度平方根归一化** (Square-root Length Normalization) 的设计动机：按样本 token 数量的平方根对损失进行归一化，防止长视频样本主导梯度。类似地，LLaVA 1.5 对视频采用更低的分辨率和更少的 token，本质上也是通过降低视频模态的“音量”来维持训练平衡。

数据混合的隐性挑战

多模态训练中的数据混合 (data mixing) 远比纯文本训练复杂。开发者不仅需要决定不同模态（文本、图像、视频）的比例，还需要在模态内部根据信息密度调整有效权重。忽视这一点会导致：

1. 视频模态因 token 数量庞大而主导损失；
2. 模型在文本推理能力上出现退化；
3. 不同模态之间的迁移学习效果不佳。

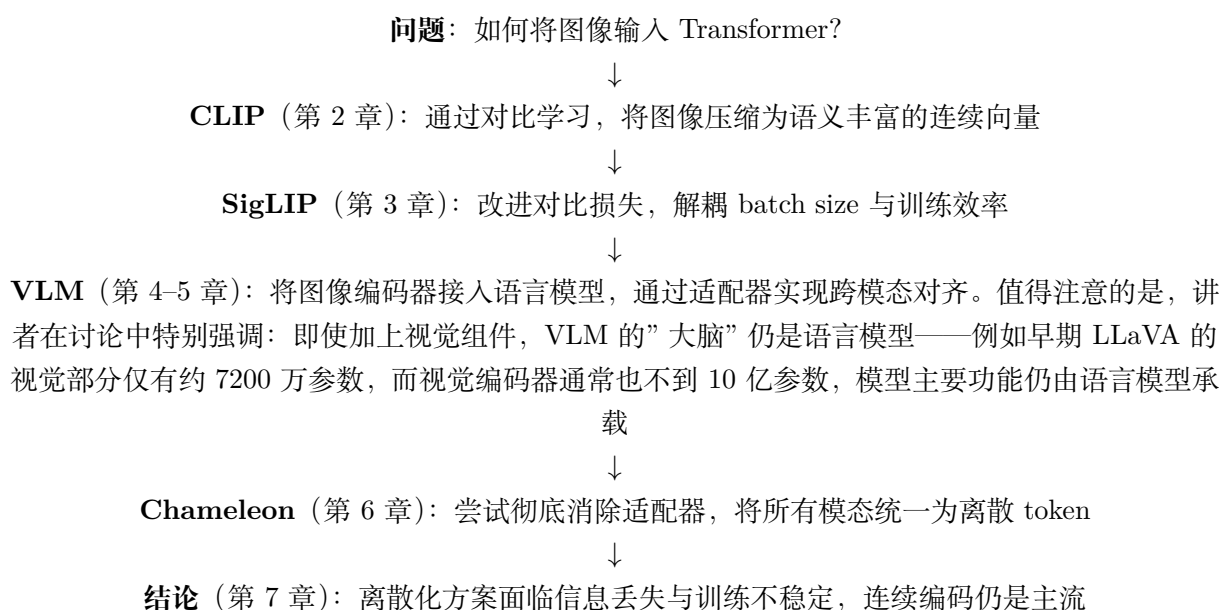
2.2 结构化提炼：核心主张与机制

基于讲者的讨论，我们可以提炼出以下核心主张：

1. **连续编码仍是当前主流**：尽管 Chameleon 等离散化方案在理论上优雅统一，但实践中连续编码器 (CLIP-like) + Transformer + 扩散模型的组合仍是最佳实践。CLIP 的概念虽已五年，但其核心思想——通过对比学习获得语义丰富的图像嵌入——依然是理解图像意义的主要方式。
2. **适配器的演进谱系**：从 LLaVA 的简单线性投影，到 LLaVA 1.5 的两层 MLP，到 Qwen-VL 的交叉注意力适配器，再到 Qwen3-VL 的**深度嵌入适配器**（将视觉特征直接注入语言模型各层残差流），适配器的设计越来越深入语言模型的内部。这一谱系的一端是”黑箱投影”，另一端是”完全无适配器”（Chameleon 的纯语言模型方案），当前前沿位于两者之间。
3. **上下文长度是多模态的关键瓶颈**：视频理解对长上下文的需求尤为迫切。Qwen3-VL 将上下文扩展至 256K，正是为了应对长视频序列的挑战。序列长度的逐步扩展（8K → 32K → 256K）已成为多模态模型训练的标准策略。
4. **位置编码的三维化演进**：从 ViT 的简单 1D 位置嵌入，到 Qwen-VL 的 2D 位置嵌入，再到 Qwen2-VL 的 MRoPE（三维：高度、宽度、时间），最后到 Qwen3-VL 的维度交错 3D RoPE——位置编码的演进清晰反映了多模态模型对空间-时间结构建模能力的不断提升。

2.3 跨章节联系与概念压缩

回顾全讲内容，我们可以将多模态对齐的技术演进压缩为一条主线：



这条主线与引言中提出的”全模态理想”形成了呼应。全模态要求任意模态输入、任意模态输出，而当前技术路径在输入侧已相对成熟 (CLIP/VLM 系列)，在输出侧则仍依赖专门模块 (扩散模型用于图像生成)。Chameleon 试图用单一模型同时解决两端，但其局限性恰恰说明了问题的难度。

2.4 具体收获与开放问题

2.4.1 具体收获

通过本讲学习，读者应当掌握：

- 多模态对齐的核心问题定义与 token 概念扩展；

- CLIP 对比学习的原理、数据构建方式及其历史意义；
- SigLIP 如何通过 sigmoid 损失解耦 batch size 与训练效率；
- VLM 的通用架构范式（冻结编码器 + 可训练适配器 + 语言模型）及 LLaVA/Qwen 系列的具体实现；
- 动态分辨率（AnyRes）、MRoPE/3D RoPE、深度嵌入适配器等关键技术创新；
- VQ-VAE 离散化方案的原理、优雅性与实际挑战；
- 编码器粒度权衡、模态信息密度差异、数据混合策略等多模态训练的核心考量。

2.4.2 开放问题

讲者在结尾处也留下了若干值得深思的开放问题：

1. **真正的原生多模态生成**：当前 VLM 只能生成文本，图像生成仍需外挂扩散模型。能否设计一个训练稳定、信息保真的统一模型，同时实现理解与生成？Chameleon 的尝试表明，这条路比预期更艰难。
2. **最优适配器设计**：从线性投影到深度嵌入，适配器的复杂度持续增加。适配器究竟应该“多深”？是否存在一个理论上的最优深度，能够在跨模态对齐与计算效率之间取得最佳平衡？
3. **模态间的信息密度量化**：我们凭直觉知道视频信息密度低于文本，但能否精确量化这一差异？是否存在一个通用的理论框架，指导多模态数据的最优混合比例？
4. **长上下文与多模态的交汇**：当上下文长度达到百万级 token 时，视频、音频等时序模态的处理将发生质变。当前 256K 的上下文是否已足够？下一代模型是否需要百万级甚至无限上下文？
5. **超越视觉-语言**：本讲聚焦于图像与文本，但全模态理想还包括音频、触觉、甚至脑信号等更多模态。这些模态的“分词器”应当如何设计？它们能否共享同一表示空间？

2.5 下一步

讲者在讲座最后鼓励学生尝试训练这些模型。对于有志于深入多模态研究的读者，建议的下一步包括：

- 复现 LLaVA 或 Qwen-VL 的训练流程，亲身体验对齐阶段与微调阶段的差异；
- 对比 CLIP 与 SigLIP 在相同数据上的训练效率，验证 SigLIP 的 batch size 解耦效果；
- 探索 VQ-VAE 的重建质量与 OCR 性能，直观感受离散化信息丢失的影响；
- 关注 Gemini、GPT-4o 等闭源模型的技术披露，验证本讲中关于“连续编码 + 扩散生成”架构猜测的准确性。

多模态对齐是一个正在快速演进的领域。从 2021 年的 CLIP 到 2024 年的 Qwen3-VL 和 Chameleon，短短三年间我们见证了从简单对比学习到复杂深度嵌入适配器的飞跃。然而，讲者的反思提醒我们：尽管技术细节日新月异，一些根本性的权衡——理解与生成的不对称、编码器粒度的选择、连续与离散的辩证关系——仍将持续塑造这一领域的未来走向。

3 CLIP：对比语言-图像预训练

3.1 历史背景与动机

回到 2021 年，自然语言处理领域已经发生了深刻变革。GPT-3 和 GPT-2 的相继问世标志着语言模型进入了“基础模型”（foundation model）时代——通过从互联网上抓取海量文本进行自监督预训练，模型能够从中提取有用信息并生成高质量内容。然而，计算机视觉领域的发展路径却截然不同：传统上，视觉模型严重依赖像 ImageNet 这样经过精心标注的大规模图像数据集，在其上训练 ResNet 等卷积网络，并配合数据增强等技术来取得优异性能。

CLIP 的核心洞见

CLIP（Contrastive Language-Image Pre-training，对比语言-图像预训练）的核心洞见在于：互联网上已经存在大量天然配对的图像与文本描述（如网页中的图片及其 surrounding text），无需昂贵的人工标注即可用于训练视觉模型。

OpenAI 的研究人员提出了一个关键问题：既然语言模型可以从混乱的互联网文本中学习，那么视觉模型是否也能利用现有的图像-文本配对数据？CLIP 正是这一思路的产物。它标志着视觉领域从“人工标注驱动”向“互联网规模弱监督数据驱动”的范式转变。

3.2 核心对比学习机制

CLIP 的训练目标在概念上非常直观。假设我们有一个包含 N 对图文配对的数据批次： $(I_1, T_1), (I_2, T_2), \dots, (I_N, T_N)$ ，我们对每张图像使用图像编码器进行编码，得到图像嵌入向量 $I \in \mathbb{R}^{N \times D}$ ；同时对每段文本使用文本编码器进行编码，得到文本嵌入向量 $T \in \mathbb{R}^{N \times D}$ 。

训练目标是：对于配对的图像 I_i 和文本 T_i ，它们的嵌入相似度应该远大于 I_i 与任何其他文本 T_j ($j \neq i$) 的相似度；反之亦然， T_i 与 I_i 的相似度也应远大于 T_i 与其他图像 I_j 的相似度。

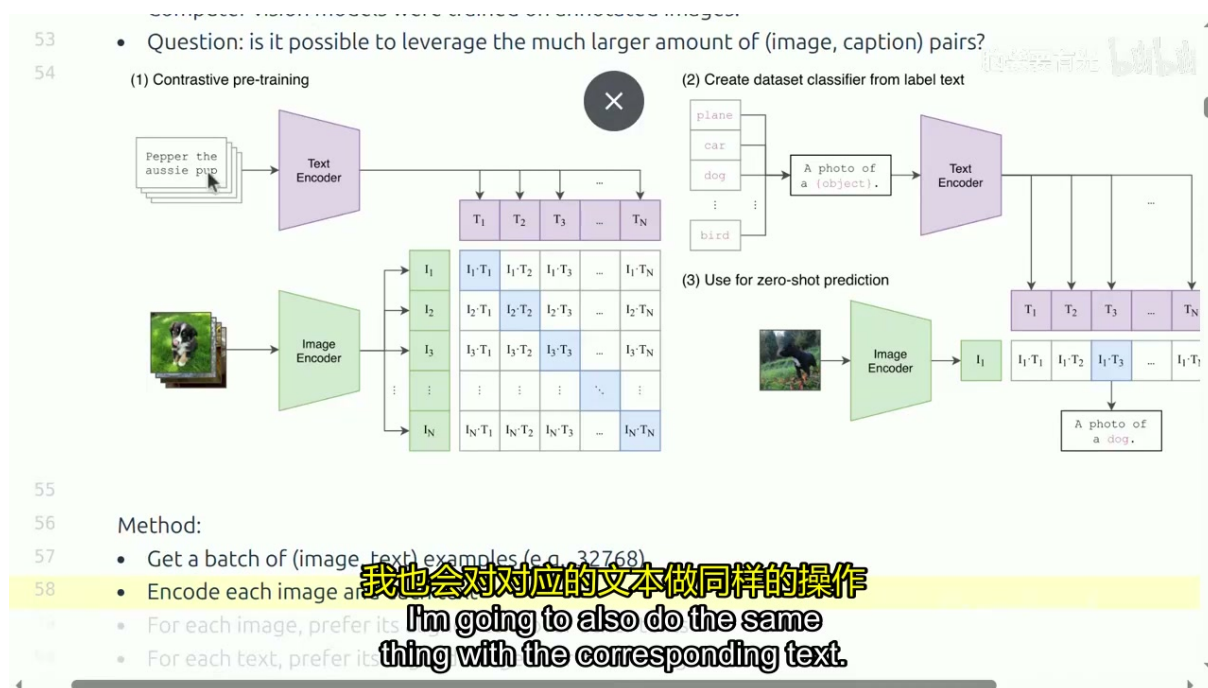


图 2: CLIP 整体架构：对比学习预训练、从标签文本构建零样本分类器、零样本预测流程

具体而言，CLIP 的损失函数包含两个方向的对称交叉熵：

1. **图像到文本** ($I \rightarrow T$)：将每个图像视为查询，在所有文本中进行分类，正确标签是对应位置的文本；
2. **文本到图像** ($T \rightarrow I$)：将每个文本视为查询，在所有图像中进行分类，正确标签是对应位置的图像。

用数学公式表达，首先计算缩放后的余弦相似度矩阵：

$$\text{sim}(I_i, T_j) = \frac{I_i \cdot T_j}{\|I_i\| \|T_j\|} \cdot \exp(\tau)$$

其中 τ 是一个可学习的温度参数 (temperature)，用于控制分布的尖锐程度。温度参数的存在使得模型可以自适应地调整相似度尺度的“软硬”程度。

然后，对称交叉熵损失为：

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2} \left[\underbrace{-\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(I_i, T_i))}{\sum_{j=1}^N \exp(\text{sim}(I_i, T_j))}}_{\text{图像} \rightarrow \text{文本}} + \underbrace{-\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\text{sim}(I_i, T_i))}{\sum_{j=1}^N \exp(\text{sim}(I_j, T_i))}}_{\text{文本} \rightarrow \text{图像}} \right]$$

从

分类的角度理解，CLIP 本质上是在进行两个 N 类别的 softmax 多类别分类任务，其中类别数量等于批次大小 N 。这也是为什么 CLIP 对批次大小极其敏感——批次越大，“负样本类别”越多，训练信号越强。

用伪代码可以更清晰地展示这一过程：

```

1  # I: [N x D] 图像嵌入矩阵
2  # T: [N x D] 文本嵌入矩阵
3  # tau: 可学习的温度参数
4
5  # L2归一化
6  I_norm = I / ||I||_2 # 按行归一化
7  T_norm = T / ||T||_2 # 按行归一化
8
9  # 计算相似度矩阵并缩放
10 logits = (I_norm @ T_norm.T) * exp(tau) # [N x N]
11
12 # 对称交叉熵损失
13 labels = arange(N) # 对角线为正样本
14 loss_i2t = cross_entropy(logits, labels, dim=1) # 图像->文本
15 loss_t2i = cross_entropy(logits, labels, dim=0) # 文本->图像
16 loss = (loss_i2t + loss_t2i) / 2

```

Listing 1: CLIP 核心损失函数伪代码

3.3 数据构建

CLIP 论文中关于数据构建的细节披露较少，但大致流程如下：

1. 从某个来源提取大量搜索查询；
2. 使用这些查询在互联网上搜索图像；
3. 收集得到的图像-文本配对。

通过这一过程，OpenAI 构建了一个包含约 **4 亿对** 图像-文本数据的数据集。然而，这个数据集并未公开发布。

C

CLIP 的原始 4 亿图文对数据集是不公开的。如果希望复现或研究 CLIP 的训练过程，需要使用替代数据源。

为了弥补这一空白，后续研究提出了 **Open CLIP**。这项工作不仅开源复现了 CLIP 的训练代码和模型权重，还基于公开可用的 LAION 数据集进行了训练。其中最重要的是 **LAION-5B** 数据集——它包含约 50 亿张带有文本描述的图像，并经过了大量的后处理（如去重、过滤低质量样本等）。有趣的是，Open CLIP 的训练过程本身也利用了 CLIP 进行数据过滤，形成了一个“自举”（bootstrapping）的过程：先用 CLIP 过滤数据，再用过滤后的数据训练 Open CLIP。

3.4 图像预处理

网络图像的分辨率和长宽比千差万别——有的细长，有的方正。神经网络通常偏好固定尺寸的输入，因此 CLIP 采用了一种简单而有效的启发式预处理流程：

1. **短边 resize**：将图像的较短边缩放到目标尺寸（CLIP 使用 336 像素）；
2. **中心裁剪**（center crop）：从 resize 后的图像中心裁剪出一个正方形区域。

C

CLIP 选择 336×336 作为训练分辨率，部分原因是考虑到 ImageNet 分类任务的特性——待分类物体通常位于图像中心，中心裁剪不会损失太多关键信息。当然，这种启发式方法只是为了快速实现，后续工作（如 LLaVA 的 AnyRes、Qwen-VL 的动态分辨率）已经提出了更优雅的方案。

值得注意的是，CLIP 的训练采用了分辨率渐进策略：训练初期使用较低分辨率以提升速度，后期再切换到更高分辨率进行微调。这种策略在计算资源有限时非常实用。

3.5 视觉编码器

CLIP 试验了多种视觉骨干网络，包括 ResNet 和当时刚刚推出的视觉 Transformer（ViT）。实验结果表明，**ViT 表现最佳**，因此当人们提及“CLIP”时，通常默认指其 ViT 版本。

3.5.1 ViT 图像分块与位置嵌入

视觉 Transformer 的处理流程如下：

1. 将输入图像切分为固定大小的 patch。原始 ViT 论文使用 16×16 的 patch，而 CLIP 使用的是 14×14 的 patch；
2. 每个 patch（包含 RGB 三个通道）被展平为一个向量，相当于 Transformer 中的一个“token”；
3. 为每个 patch 添加可学习的位置嵌入（positional embedding），类似于语言模型中的处理方式；
4. 将 patch 序列输入标准 Transformer 编码器进行处理。

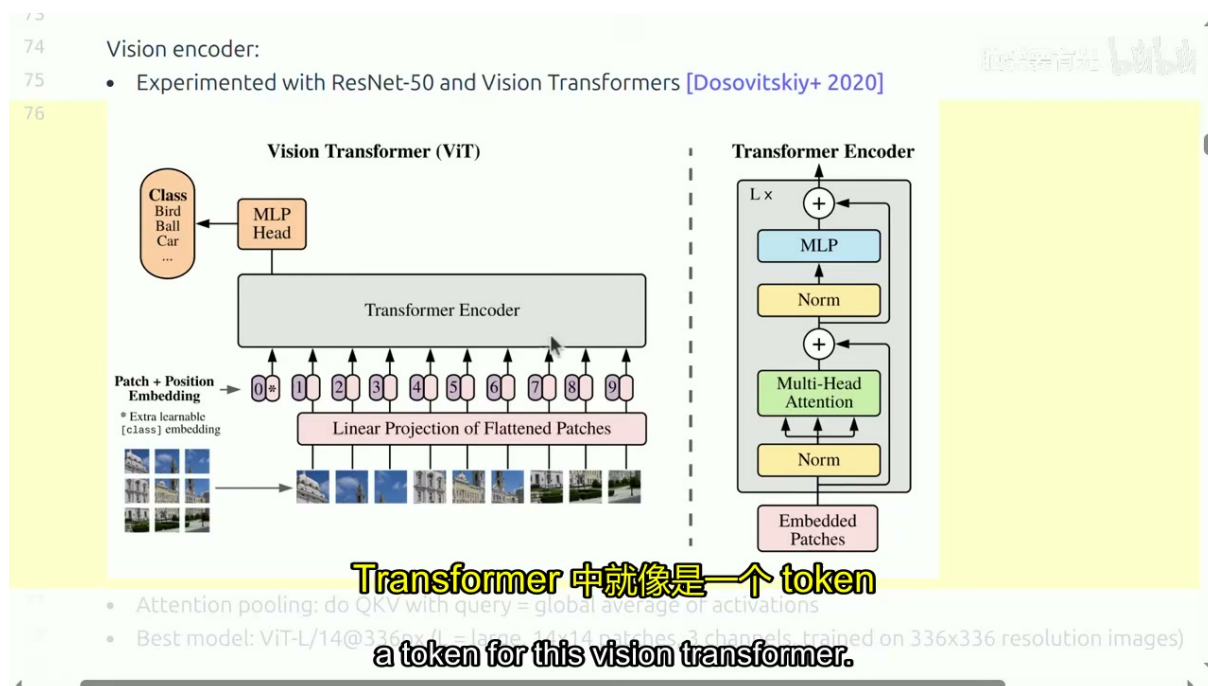


图 3: Vision Transformer (ViT) 架构：图像切分为 patch、添加位置嵌入、经 Transformer 编码器处理

对于 336×336 分辨率的图像，使用 14×14 的 patch，会得到 $(336/14) \times (336/14) = 24 \times 24 = 576$ 个 patch token。

3.5.2 Attention Pooling

标准 ViT 在分类任务中通常会在序列末尾添加一个可学习的 [class] token，或者对所有 patch token 进行全局平均池化来获取图像级别的表示。CLIP 采用了一种更精细的替代方案——**注意力池化** (Attention Pooling)：

1. 首先计算所有 patch 特征的全局平均；
2. 将这个全局平均向量作为查询（query），与每个位置的键（key）和值（value）进行额外的注意力计算；
3. 输出一个聚合后的单一向量作为图像表示。

注

注意力池化相比简单平均的优势在于：可学习的查询向量能够自适应地关注图像中更具信息量的区域，而不是对所有 patch 一视同仁。这对于图像-文本对齐任务尤为重要，因为文本描述可能只涉及图像中的部分语义内容。

CLIP 论文中表现最好的模型配置是 **ViT-L/14**：这是一个大型 ViT (Large, 约 24 层)，使用 14×14 的 patch，在 336×336 分辨率下训练。

关

于位置嵌入，有同学问到是否可以采用二维位置编码（考虑图像的空间结构）。CLIP 论文确实尝试了二维版本的位置嵌入，但实验发现对最终分类性能影响不大。不过，这并不意味着空间位置信息不重要——后续更复杂的模型（如 Qwen-VL 的 2D 位置嵌入、MRoPE）将证明，当任务需要更精细的空间理解时，显式的空间编码变得至关重要。

3.6 文本编码器

CLIP 的文本编码器是一个标准的 GPT 风格的 Transformer 解码器（注意：不是 BERT 式的双向编码器）。由于研发团队与 GPT 系列相同，他们沿用了成熟的 GPT 架构设计。

为了从变长文本序列中获得固定长度的单一向量表示，CLIP 采用了以下策略：

1. 在序列开头添加特殊的 [BOS] (Beginning of Sequence) token；
2. 在序列末尾添加 [EOS] (End of Sequence) token；
3. 取 [EOS] token 对应的最终隐藏状态作为整个文本序列的表示向量。

3.7 零样本分类

CLIP 在 2021 年引起广泛关注的核心原因之一，是其在 ImageNet 基准测试上展现了惊人的**零样本** (zero-shot) 分类能力。

零

样本 CLIP 在 ImageNet 上的性能超过了基于 120 万张 ImageNet 标注图像训练的传统监督模型。考虑到 ImageNet 的 120 万张标注需要大量亚马逊 Mechanical Turk 外包工人耗费大量时间完成，而 CLIP 仅利用互联网上的原生图文数据就能达到甚至超越这一水平，这一结果具有里程碑意义。

零样本分类的原理非常优雅：

1. 给定一张待分类的图像，使用 CLIP 图像编码器获取其嵌入向量；
2. 将所有候选类别标签转换为自然语言描述（如将“狗”转换为“a photo of a dog”）；
3. 使用文本编码器获取每个描述的嵌入向量；
4. 计算图像与所有文本描述之间的余弦相似度；

5. 选择相似度最高的文本描述对应的类别作为预测结果。

这种方法的灵活性在于：通过改变文本提示（prompt）的措辞，模型可以适应不同的分类任务，而无需任何任务特定的微调。例如，可以将”a photo of a {}”用于通用图像分类，或将”a sketch of a {}”用于草图识别。

3.8 噪声鲁棒性分析

一个自然的问题是：互联网上的图文配对数据质量参差不齐，这种噪声会不会摧毁训练？

网

络图文数据的噪声来源包括：

- 文本描述并不总是准确描述图像内容（例如，图中有一只狗，但描述中并未提及”狗”）；
- 网页抓取过程中可能引入无关文本；
- 同一批次中可能存在语义相似的负样本（如另一张也有狗的图片）。

CLIP 的对比学习框架对噪声具有出人意料的鲁棒性：

1. **语义相似负样本不是问题**：即使另一张图片也包含狗，只要描述不同，模型仍然能够区分；
2. **噪声的统计性质**：虽然单个样本可能噪声很大，但在 4 亿规模的巨大数据集中，正确的统计信号仍然占主导地位；
3. **数据过滤的必要性**：如果完全不加筛选地从网络抓取任意图像和文本，噪声可能过大。因此实际训练中仍需要一定程度的过滤和清洗。

噪

声鲁棒性并不意味着数据质量不重要。从网络上获取的原始图文对仍然需要过滤，以去除极端噪声样本。Open CLIP 和 LAION-5B 的工作就包含了大量的数据清洗和后处理步骤。

3.9 对比实验：预测文本 vs 对比学习

CLIP 的研究团队还进行了一项颇具启发性的对比实验：他们不是用对比学习目标，而是尝试让模型**从图像中直接预测文本**（即训练一个图像到文本的生成模型，类似 image captioning 任务）。

实验结果出人意料：

- 使用更强大的生成模型架构（如更大的 Transformer decoder）进行文本预测，其效率反而**更低**；
- 对比学习（CLIP 的方式）在相同的计算预算下取得了更好的 ImageNet 准确率；
- 精确建模文本描述的词序列对于学习粗略的图像语义表示并不重要。

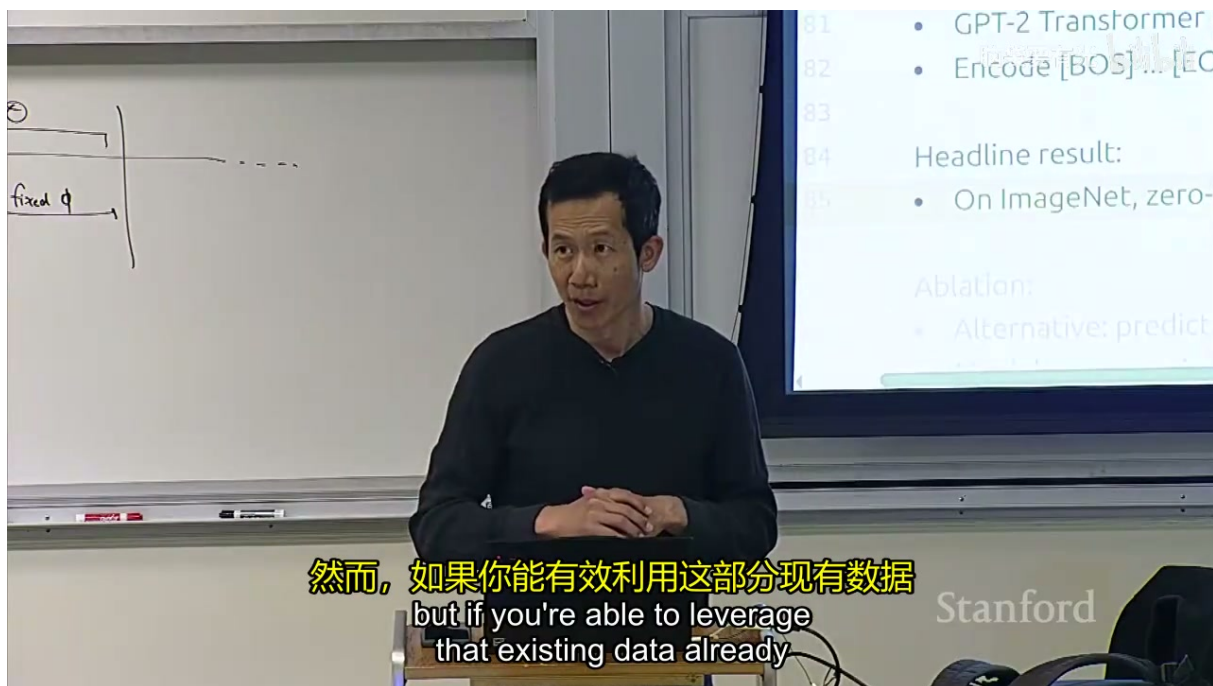


图 4: CLIP 消融实验：对比 Bag of Words、Contrastive、Prediction、Transformer LM 等方法的计算效率与 ImageNet 准确率

这

一实验揭示了一个深刻的洞见：对于图像语义理解任务，学习一个能够将图像和文本映射到共享嵌入空间的判别式表示，比精确建模文本生成的生成式目标更加高效。因为图像-文本对齐只需要粗粒度的语义匹配，而不需要细粒度的词级预测。

3.10 CLIP 的局限性

在总结 CLIP 之前，有必要指出其关键的技术局限性——这些局限性直接启发了后续 SigLIP 等改进工作：

1. **对批次大小的极端依赖**：CLIP 需要极大的批次大小（约 32,000）才能有效训练。如果批次大小为 1 或 10，训练几乎不可能收敛；
2. **softmax 的全局性**：softmax 操作需要在整个批次上进行归一化，这使得损失函数与批次大小紧密耦合，难以在分布式环境中高效扩展；
3. **固定分辨率**：336 × 336 的固定输入尺寸限制了模型处理高分辨率图像或任意长宽比图像的能力；
4. **粗粒度语义**：CLIP 的设计目标偏向图像分类，因此其视觉表示捕获的是高层语义而非细粒度细节（这对后续 OCR 等任务构成了挑战）。

LIP 的 batch size 依赖性意味着：改变 batch size 就等同于改变了损失函数本身。这与标准语言模型训练形成鲜明对比——在语言模型中，每个序列的梯度计算是独立的，batch 只是对独立梯度做聚合。

3.11 本章小结

本章深入讲解了 CLIP 模型的核心机制：

- **历史动机**：2021 年语言模型已进入基础模型时代，而视觉仍依赖 ImageNet 等人工标注数据集；
- **核心机制**：通过对比学习将图像和文本映射到共享嵌入空间，使用对称 softmax 交叉熵损失和可学习的温度参数；
- **数据**：4 亿互联网图文对（不公开），后续 Open CLIP 使用公开的 LAION-5B 数据集复现；
- **预处理**：短边 resize 到 336 + 中心裁剪的启发式方法；
- **视觉编码器**：ViT-L/14，使用 14×14 patch、注意力池化替代全局平均池化；
- **文本编码器**：GPT 风格 Transformer，取 [EOS] token 作为序列表示；
- **零样本能力**：无需微调即可在 ImageNet 上超越传统监督模型，标志着视觉模型进入基础模型时代；
- **噪声鲁棒性**：大规模对比学习对网络数据的噪声具有天然鲁棒性；
- **对比实验**：预测文本的生成式目标效率低于对比学习的判别式目标。

CLIP 为后续所有视觉语言模型（VLM）奠定了视觉编码器的基础。然而，其对极大 batch size 的依赖和训练效率问题，促使研究者寻找更高效的替代方案——这正是下一章 SigLIP 将要解决的核心问题。

4 SigLIP：解耦批大小的对比学习

在上一节中，我们深入了解了 CLIP 的核心机制及其历史意义。CLIP 通过对比学习将图像和文本映射到共享的嵌入空间，为现代视觉语言模型奠定了基础。然而，CLIP 在实际训练中存在一个显著的技术瓶颈：它对极大的批大小（batch size）有强烈依赖。OpenAI 的 CLIP 使用了约 32,000 的批大小，这意味着在分布式训练中，所有设备上的样本必须共同参与 softmax 归一化，导致计算和通信成本高昂，且难以在资源受限的环境下扩展。本节将介绍 Google 提出的 SigLIP (Sigmoid Loss for Language Image Pre-training)，它通过将 CLIP 的 softmax 多类别分类目标替换为逐对 sigmoid 二分类目标，优雅地解决了这一问题。

4.1 从 Softmax 到 Sigmoid: 损失函数的重新设计

CLIP 的核心损失函数是对称的交叉熵损失。对于一批 N 个图文对，图像编码矩阵 $I \in \mathbb{R}^{N \times D}$ 和文本编码矩阵 $T \in \mathbb{R}^{N \times D}$ ，CLIP 计算一个 $N \times N$ 的相似度矩阵，然后对每一行（图像到文本）和每一列（文本到图像）分别做 softmax 多类别分类。这意味着，对于第 i 个图像，它必须在全部 N 个文本描述中找出正确匹配的那一个——本质上是一个 N 路分类问题。

C

LIP 的损失函数与批大小 N 紧密耦合：当 N 改变时，softmax 的分母（所有负样本的指数和）随之改变，损失函数的形态也随之变化。因此，小 batch（如 $N < 16K$ ）下的训练效果会显著下降。

SigLIP 的核心改进在于将这一 N 路分类问题简化为逐对的二分类问题。对于相似度矩阵中的每一个元素 $\text{sim}(I_i, T_j)$ ，SigLIP 不再要求它在整行或整列中“胜出”，而是直接判断图像 I_i 和文本 T_j 是否匹配：

- 对角线元素 ($i = j$)：正样本，标签 $y_{ij} = +1$
- 非对角线元素 ($i \neq j$)：负样本，标签 $y_{ij} = -1$

损失函数变为所有 N^2 个配对的 log-sigmoid 损失之和：

$$\mathcal{L}_{\text{SigLIP}} = - \sum_{i=1}^N \sum_{j=1}^N \log \sigma(y_{ij} \cdot \text{sim}(I_i, T_j))$$

其中 $\sigma(\cdot)$ 是 sigmoid 函数， $\text{sim}(I_i, T_j)$ 是归一化后的缩放余弦相似度（与 CLIP 相同，包含可学习的温度参数 τ ）。

S

igmoid 损失的优势在于其“局部性”：每个图文对的损失只依赖于该对自身的相似度，而不需要知道整个 batch 中所有其他样本的相似度。这使得损失函数的形态不再随 batch size 变化——小 batch 下每个样本的期望损失与大 batch 下相同，只是方差更大。

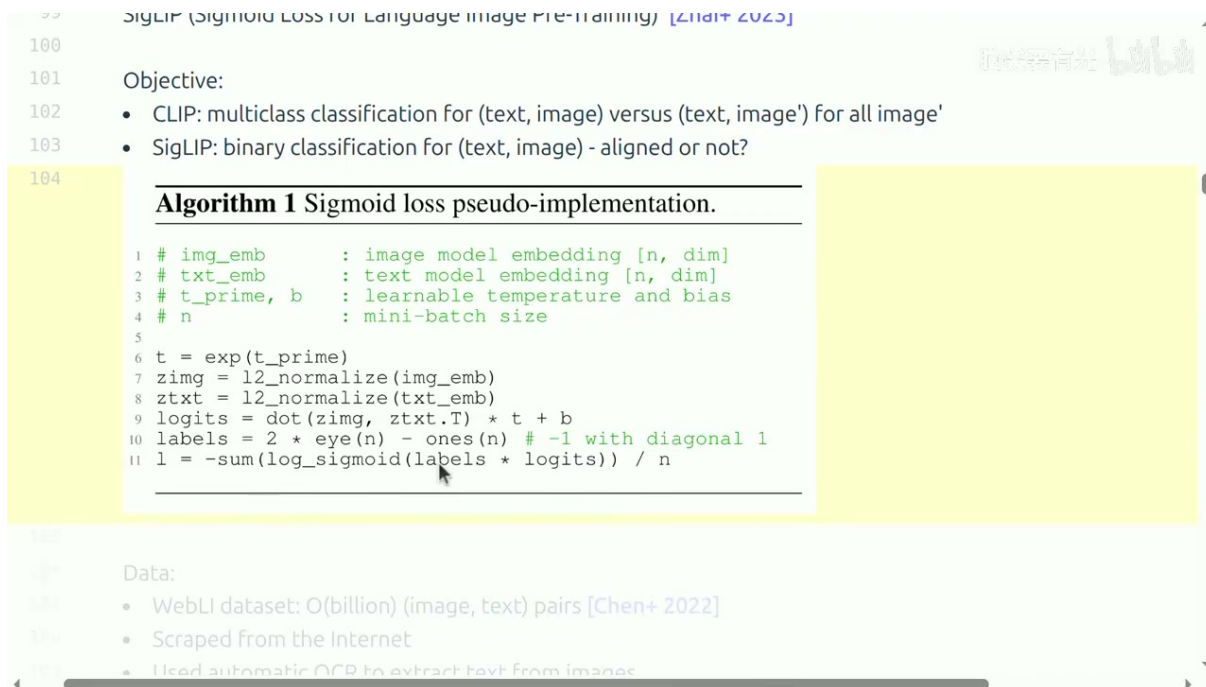


图 5: SigLIP 目标函数：将 CLIP 的 softmax 多类别分类替换为逐对 sigmoid 二分类，对角线标签为 +1，非对角线为 -1

4.2 无需复杂负采样策略的简洁性

在对比学习的研究历史中，负样本的选择策略一直是一个关键课题。早期的自监督对比学习方法（如 SimCLR）发现，负样本的质量直接影响表示学习的效果：如果负样本过于简单（与正样本差异极大），模型学不到有用的判别信息；如果负样本过于困难，又可能导致训练不稳定。

SigLIP 的论文指出，至少在最初的版本中，他们并未采用复杂的负采样策略。所有非对角线元素统一被视为负样本，直接最小化 sigmoid 损失即可。这与 CLIP 形成了有趣的对比：CLIP 通过 softmax 隐式地实现了“困难负样本挖掘”——因为 softmax 会自然地将更多梯度分配给那些与正样本相似度较高的负样本；而 SigLIP 则通过更简洁的逐对二分类，将这一复杂性完全消除。

S

igLIP 的简洁性并不意味着它在所有场景下都优于 CLIP。当 batch size 足够大时（如 $N \approx 32K$ ），CLIP 的 softmax 机制能够利用更多的负样本信息，其性能与 SigLIP 相当。SigLIP 的真正优势在于小 batch 场景和分布式训练效率。

4.3 分布式训练中的环状通信

CLIP 的 softmax 损失要求在整个 batch 上计算归一化，这意味着在数据并行训练中，所有设备必须在每次前向传播后交换完整的图像和文本嵌入。当 batch size 达到 32,000 且嵌入维度 D 较大时，这种全互联通信（all-to-all communication）成为严重的瓶颈。

SigLIP 的逐对 sigmoid 损失虽然也需要计算所有 N^2 个配对的相似度，但它允许一种更高效的分布式策略——**环状通信**（ring communication）。其工作流程如下：

1. 假设有 K 个计算设备，每个设备本地保存 N/K 个图文对。

2. 在第一次遍历中，每个设备计算所有本地图像与本地文本之间的损失（对应相似度矩阵的本地对角块）。
3. 然后，设备之间按环形拓扑传递文本嵌入：设备 1 将文本发送给设备 2，设备 2 发送给设备 3，依此类推，设备 K 发送给设备 1。
4. 接收到新的文本嵌入后，每个设备计算本地图像与这些外来文本之间的负样本损失（对应非对角块）。
5. 重复上述传递过程，直到每个设备都遍历过所有其他设备的文本嵌入，即所有 N^2 个非对角条目都被计算完毕。

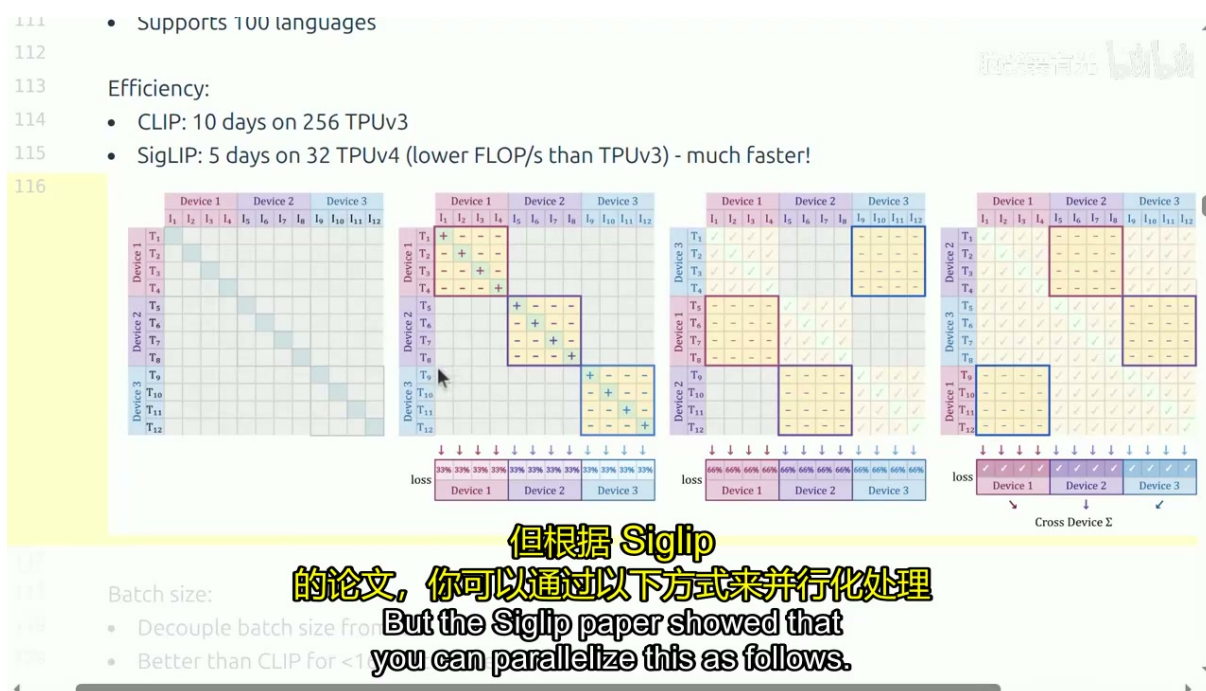


图 6: SigLIP 分布式训练中的环状通信策略：各设备按环形拓扑传递文本嵌入，逐步计算所有 N^2 个非对角负样本损失

这种环状通信的优势在于：

- **通信量可控**：每轮通信只传递文本嵌入矩阵的一个子块，而非整个 $N \times D$ 矩阵。
- **计算与通信重叠**：在传递下一轮文本嵌入的同时，可以并行计算当前轮的损失。
- **无需全局同步**：避免了传统 all-reduce 操作中所有设备同时等待的同步开销。

4.4 训练效率对比：CLIP vs SigLIP

SigLIP 论文提供了令人印象深刻的训练效率对比数据。CLIP 在 256 个 TPU v3 芯片上训练了约 10 天；而 SigLIP 仅使用 32 个 TPU v4 芯片，训练时间缩短到 5 天。

训

训练效率的显著提升并非主要来自硬件升级（TPU v4 的单卡算力实际上比 v3 慢约 60%），而是来自 SigLIP 在算法层面的优化：更小的 batch 即可达到相似效果、更高效的分布式通信模式、以及更简洁的损失计算。

值得注意的是，SigLIP 论文也指出了 batch size 的”临界值”现象：当 batch size 增加到约 32,000 时，性能收益趋于饱和——这是对比学习负样本数量的有效上限。但 SigLIP 的真正优势在于，即使 batch size 小于 16,000（远小于 CLIP 所需的 32,000），其训练效果依然优于 CLIP。这意味着 SigLIP 在资源受限环境下（无法使用超大 batch 时）仍能保持竞争力，而 CLIP 在此条件下性能显著下降。

4.5 本章小结

SigLIP 通过对损失函数的重新设计，将 CLIP 的 softmax 多类别分类转化为逐对 sigmoid 二分类，实现了 batch size 与损失函数的解耦。这一改动带来了三个关键优势：

1. **小 batch 有效性**：即使在 $N < 16K$ 的条件下，SigLIP 仍能保持稳定的训练效果，而 CLIP 在此条件下性能显著下降。
2. **分布式通信效率**：环状通信策略将全局 all-to-all 通信转化为局部环形传递，大幅降低了多设备训练中的通信瓶颈。
3. **训练成本降低**：SigLIP 以远少于 CLIP 的硬件资源（32 TPU v4 vs 256 TPU v3）和训练时间（5 天 vs 10 天）达到了可比甚至更优的性能。

SigLIP 的改进虽然聚焦于训练效率，但其核心思想——将全局归一化问题分解为局部二分类问题——对后续的多模态模型训练产生了深远影响。现在，我们已经拥有了高效的图像编码器（CLIP 和 SigLIP），接下来的问题是如何将这些编码器与大型语言模型结合，构建能够真正”理解”图像并生成自然语言描述的视觉语言模型。

5 视觉语言模型架构：LLaVA 与 Qwen-VL

在掌握了图像编码器（CLIP/SigLIP）的基础上，本节将探讨如何将这些视觉组件与大型语言模型（LLM）融合，构建能够同时接收图像和文本输入、并生成文本输出的视觉语言模型（Vision Language Model, VLM）。我们将重点分析两个具有代表性的开源工作：LLaVA 系列和 Qwen-VL，它们分别代表了 VLM 发展的两条重要路径。

5.1 VLM 的通用架构范式

现代 VLM 普遍遵循一个三段式架构：

1. **视觉编码器**（Vision Encoder）：通常使用预训练的 CLIP ViT，将输入图像编码为一系列视觉嵌入向量。该组件在 VLM 训练中通常保持冻结，以保留预训练阶段学到的丰富视觉表示。
2. **投影器/适配器**（Projector / Adapter）：连接视觉编码器和语言模型的可训练模块，负责将视觉嵌入映射到语言模型的输入空间。早期的 VLM 使用简单的线性投影（如 LLaVA 的矩阵 W ），后续工作逐渐引入更复杂的结构（如 MLP、交叉注意力等）。

3. **语言模型 (Language Model)**: 通常使用预训练的大型语言模型 (如 Vicuna、Qwen 等), 负责接收融合后的视觉-文本 token 序列并生成输出文本。在训练的不同阶段, 语言模型可能被冻结或微调。

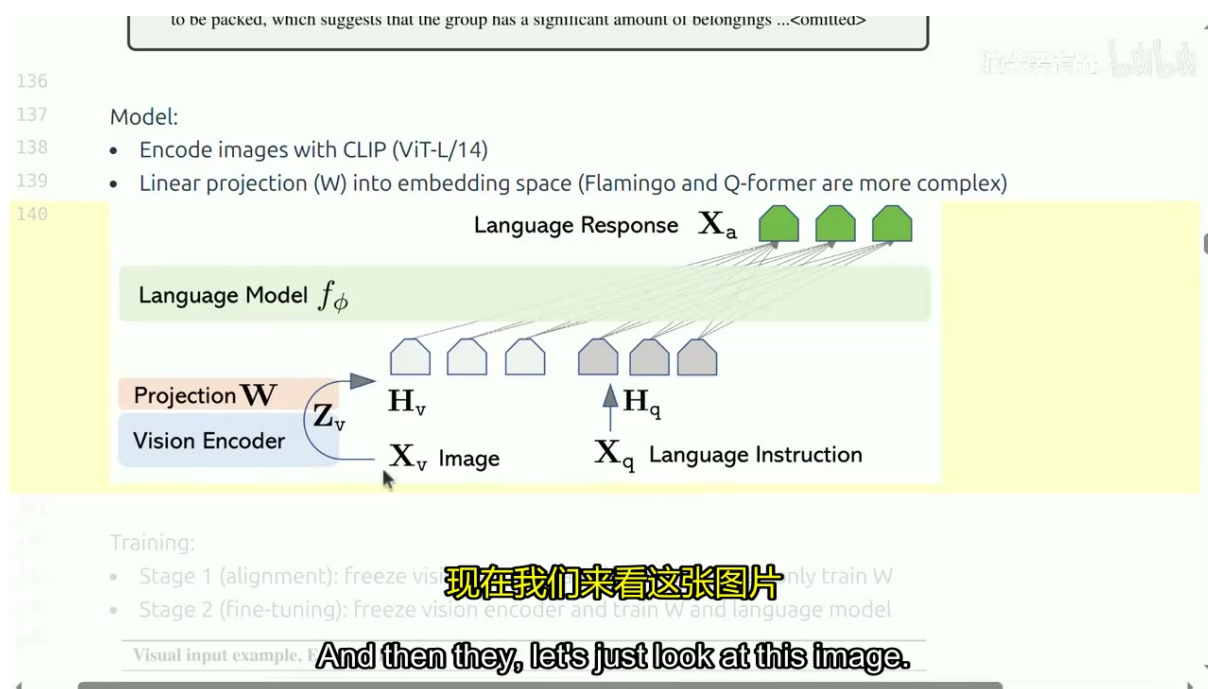


图 7: VLM 通用架构: 视觉编码器 (CLIP ViT) → 投影器/适配器 → 语言模型 (LLM), 图像与语言指令的交互流程

这

种”冻结视觉编码器 + 可训练投影器 + 冻结/微调语言模型”的范式之所以成为主流, 核心原因在于参数效率: 视觉编码器和语言模型各自已经在大规模数据上预训练, 包含了丰富的世界知识和表示能力。VLM 训练的目标不是从头学习视觉或语言表示, 而是学习如何将两者对齐到同一语义空间。

5.2 LLaVA: 两阶段训练与合成数据

LLaVA (Large Language and Vision Assistant) 发表于 2023 年, 是早期开源 VLM 中最具影响力的工作之一。它的核心设计选择包括:

5.2.1 模型组件

LLaVA 使用 CLIP ViT-L/14 作为视觉编码器, Vicuna (基于 LLaMA 微调的开源对话模型) 作为语言模型。视觉编码器的输出通过一个可训练的线性投影矩阵 W 映射到语言模型的嵌入空间。

5.2.2 两阶段训练策略

LLaVA 的训练分为两个明确阶段:

第一阶段: 对齐阶段 (Alignment Stage)

在此阶段，视觉编码器和语言模型均保持冻结，仅训练投影矩阵 W 。训练数据使用 CC3M 等大规模图文对数据集（约 60 万样本）。这一阶段的目标非常明确：让视觉嵌入经过 W 变换后，在语言模型的嵌入空间中“看起来像”自然语言 token 的嵌入。如果 W 是随机初始化的，视觉嵌入将不会对应任何有意义的语言表示；通过对齐训练， W 学习将视觉语义映射到语言语义空间。

第二阶段：微调阶段（Fine-tuning Stage）

在此阶段，视觉编码器保持冻结，但投影矩阵 W 和语言模型均被微调。LLaVA 的创新之处在于使用了高质量的**合成数据**来驱动这一阶段的训练。

5.2.3 合成数据生成

LLaVA 的数据构建基于 MS COCO 图像标注数据集。研究人员利用 GPT-4 的文本能力，基于 COCO 的人工标注（图像描述、检测到的对象列表等）生成三种类型的训练样本：

- **对话数据**：模拟用户与助手关于图像内容的问答对话
- **详细描述**：对图像内容的丰富文本描述
- **复杂推理**：需要多步推理才能回答的问题（如“图中有什么不寻常之处？”）

总计约 158,000 个样本。这种“用强大的闭源模型（GPT-4）生成数据来训练开源模型”的范式后来被称为**知识蒸馏**（Knowledge Distillation），在 VLM 领域被广泛采用。

L

LaVA 论文中展示了一个经典示例：输入一张“有人在小型货车后部熨烫衣物”的图片，用户问“这张图片中有什么不寻常之处？”，模型能够回答“通常不会在小型货车的后部熨烫衣物”。这一例子展示了 VLM 不仅需要识别物体，还需要理解场景中的语义异常——这是单纯的图像分类模型无法完成的任务。

5.3 LLaVA 1.5 的改进：AnyRes 与多模态扩展

LLaVA 1.5 在原始 LLaVA 的基础上引入了多项关键改进，使其能够处理更丰富的输入类型和更高的分辨率。

5.3.1 AnyRes：任意分辨率处理

CLIP 的视觉编码器要求固定尺寸的输入（如 336×336 像素）。对于 OCR 等需要保留精细细节的任务，这种固定分辨率严重限制了模型性能——如果将一张包含小字体的文档图片缩放到 336×336 ，文字将变得无法辨认。

LLaVA 1.5 提出的 **AnyRes**（Any Resolution）方案核心思想非常直观：

1. 将高分辨率图像切分为多个符合编码器输入尺寸的块（如多个 336×336 的 patch）。
2. 对每个块分别通过视觉编码器进行编码，得到各自的视觉嵌入序列。
3. 将所有块的嵌入序列按空间位置拼接起来，形成完整的图像表示。
4. 如果块的数量过多导致 token 序列过长，则通过下采样减少块的数量。

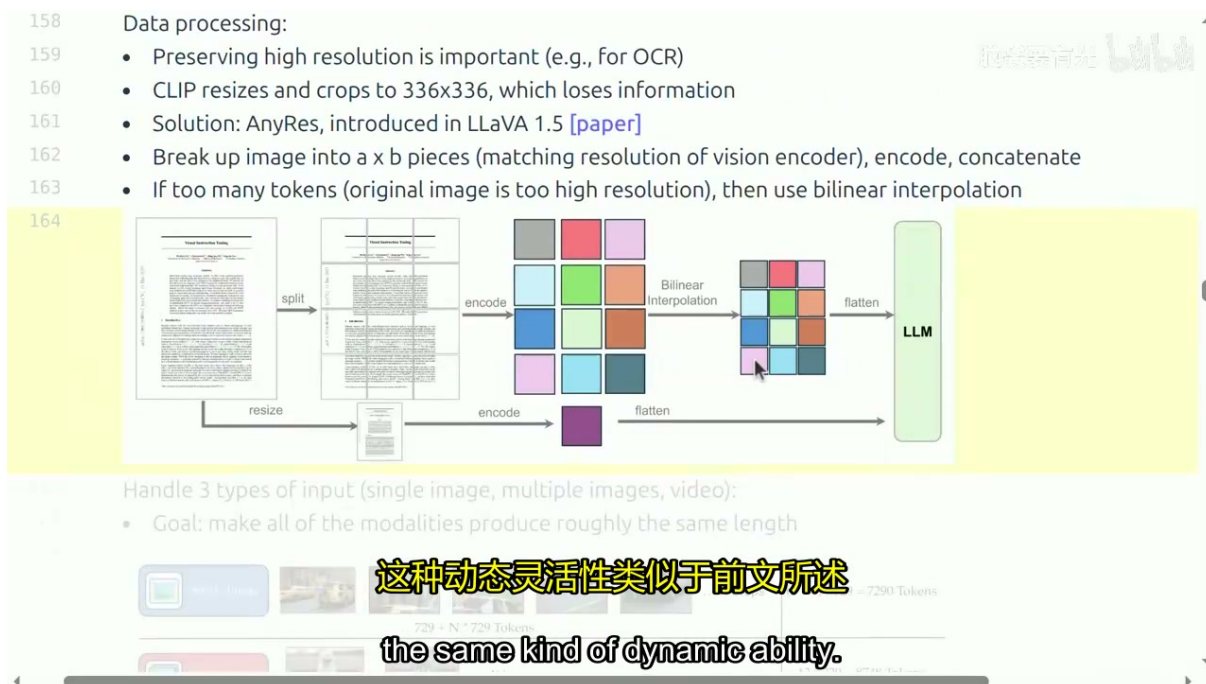


图 8: AnyRes 任意分辨率处理: 将高分辨率图像切分为多个 336×336 块分别编码, 再拼接为完整表示输入语言模型

此外, LLaVA 1.5 还引入了一个**全局编码路径**: 在切分之前, 先将整张图像下采样到 336×336 进行编码, 得到一个全局概览的嵌入。这个全局嵌入与各个局部块的嵌入拼接在一起, 使模型同时掌握图像的整体结构和局部细节。

5.3.2 多图像与视频支持

LLaVA 1.5 将架构扩展到多图像和视频输入:

- **多图像**: 多张图像分别编码后, 其嵌入序列按顺序拼接输入语言模型。
- **视频**: 将视频视为采样帧的序列 (例如每段视频取 32 帧), 每帧作为一张图像进行编码。为了控制上下文长度, 视频帧使用更低的分辨率和更少的 token。

5.3.3 模态自适应的数据处理

LLaVA 1.5 针对不同输入类型采用了差异化的处理策略:

- **单张图像**: 使用高分辨率, 进行多次裁剪 (最多 9 个局部块), 保留尽可能多的细节。
- **多张图像**: 使用基础分辨率, 避免 token 数量爆炸。
- **视频**: 使用更低分辨率和更少的每帧 token, 因为视频帧之间存在大量冗余信息。

这种模态自适应策略的核心动机是: 不同模态的信息密度不同, 不应让高 token 数量的模态 (如长视频) 主导训练过程。

5.3.4 三阶段训练与跨模态迁移

LLaVA 1.5 将训练扩展为三阶段：

1. **对齐阶段**：仅训练投影器，使用大规模低质量图文对。
2. **高质量数据阶段**：训练投影器和语言模型，使用经过精心筛选的高质量数据。
3. **下游任务阶段**：针对特定任务（如 OCR、表格问答、图表推理）进行微调。

LLaVA 1.5 论文中报告了一个有趣的**跨模态迁移**现象：模型仅在单张图像的 OCR 数据上训练，却能在测试时处理包含多张图像的复杂任务（如同时看一张表格图和一张图表图，并对它们之间的关系进行推理）。这说明 VLM 学到的视觉-语言对齐能力具有一定的泛化性，不严格局限于训练时的具体任务格式。

5.3.5 视觉提示扩展

LLaVA 1.5 还探索了**视觉提示**（Visual Prompting）的概念：在图像上叠加标记（如圆圈、箭头）来引导模型关注特定区域。这种提示方式从静态图像扩展到视频场景，例如用户可以在视频帧中圈出一名球员，要求模型描述该球员在视频中的行为。

5.4 Qwen-VL：交叉注意力与 2D 位置感知

Qwen-VL 是阿里巴巴通义千问团队于 2023 年发布的视觉语言模型。与 LLaVA 相比，Qwen-VL 在适配器设计和位置编码方面采用了不同的技术路线。

5.4.1 模型架构

Qwen-VL 使用 Open CLIP（CLIP 的开源复现版本）作为视觉编码器。与 LLaVA 的关键区别在于适配器的设计：

- LLaVA 使用简单的线性投影或两层 MLP。
- Qwen-VL 使用**交叉注意力适配器**（Cross-attention Adapter），将视觉嵌入通过交叉注意力机制映射到固定长度（256 个 token）。

交叉注意力适配器引入了额外的可学习查询向量，这些查询向量与视觉嵌入做注意力计算，从而将变长的视觉嵌入序列压缩为固定长度的表示。同时，Qwen-VL 在适配器中融入了 **2D 位置嵌入**，使模型能够感知图像中各个 patch 的空间位置关系。

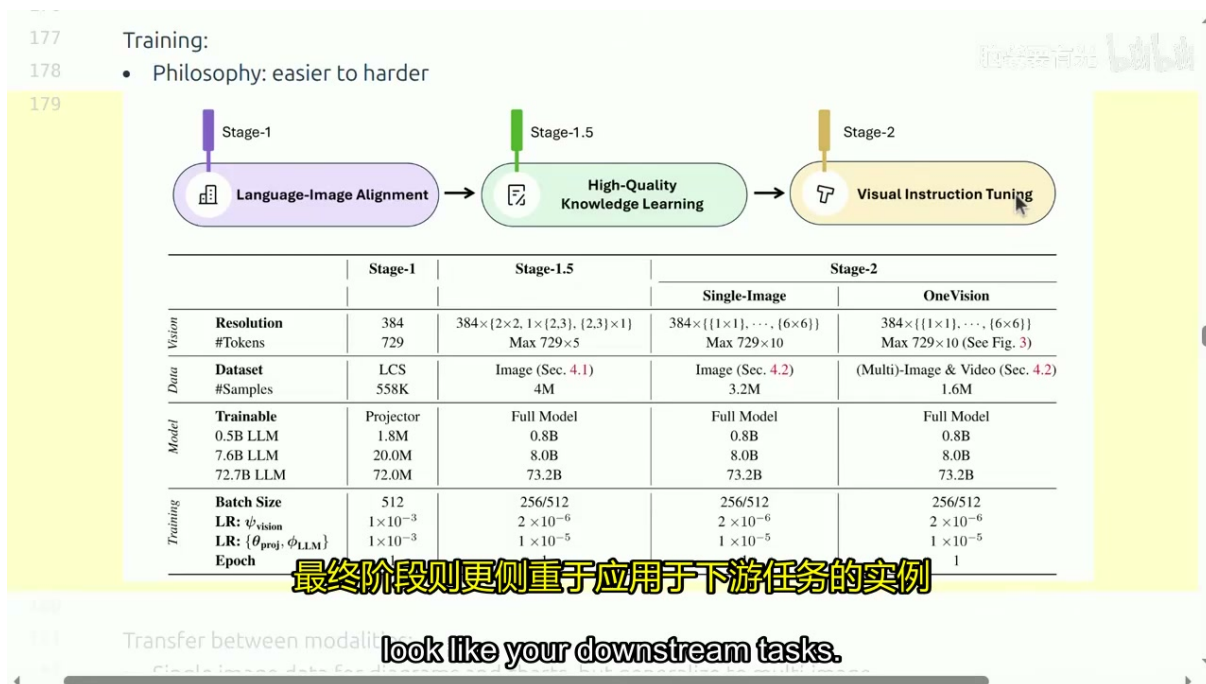


图 9: Qwen-VL 训练三阶段流程：预训练 → 多任务预训练 → 监督微调，各阶段的训练配置与目标

5.4.2 特殊 Token 设计

Qwen-VL 引入了一组特殊 token 来支持多样化的输出格式：

- ：图像起始/结束标记
- <box>：边界框标记，用于输出目标检测的坐标
- <caption>：图像描述标记

这种设计使 Qwen-VL 不仅能生成自由文本，还能输出结构化的边界框信息，支持目标定位、OCR 等需要精确空间坐标的任务。

5.4.3 三阶段训练

Qwen-VL 的训练同样分为三个阶段，但与 LLaVA 在细节上有差异：

1. **预训练阶段**：使用约 14 亿样本的大规模低质量数据。与 LLaVA 不同的是，Qwen-VL 在此阶段同时训练视觉编码器和适配器，而语言模型保持冻结。这使得视觉编码器能够更好地适应后续的多模态任务。
2. **多任务阶段**：使用高质量的特定任务数据（如 VQA、图表问答、OCR 等），训练所有参数（视觉编码器、适配器、语言模型）。
3. **指令微调阶段**：使用指令格式的对话数据，冻结视觉编码器，微调适配器和语言模型，使模型具备遵循人类指令的能力。

wen-VL 特别强调了中文能力和代码理解能力。除了处理自然语言图像描述外，它还能理解包含代码截图的图像、回答中文视觉问答、以及输出中文边界框描述。这种多语言、多任务的设计使其在实际应用中具有更广泛的适用性。

5.5 LLaVA 与 Qwen-VL 的对比

表 1: LLaVA 与 Qwen-VL 的关键对比

特性	LLaVA	Qwen-VL
视觉编码器	CLIP ViT-L/14	Open CLIP
语言模型	Vicuna	Qwen
适配器类型	线性投影/MLP	交叉注意力 + 2D 位置嵌入
视觉编码器训练	完全冻结	预训练阶段可训练
特殊输出 token	无	图像、边界框、描述标签
分辨率处理	AnyRes (切分拼接)	固定分辨率
多语言支持	以英文为主	中英文并重

两个模型代表了 VLM 早期发展的两种思路：LLaVA 追求简洁和可扩展性（AnyRes、视觉提示），而 Qwen-VL 则在适配器结构（交叉注意力）和输出格式（边界框 token）上做了更多探索。它们共同确立了 VLM 的基本训练范式——分阶段、冻结大部分参数、依赖高质量合成数据——这些原则在后续的 Qwen2-VL、Qwen3-VL 等工作中得到了继承和深化。

5.6 本章小结

本节介绍了视觉语言模型的通用架构和两种代表性实现：

1. **VLM 通用范式**：冻结视觉编码器 + 可训练投影器/适配器 + 冻结/微调语言模型。这种三段式架构的参数效率是其成功的关键。
2. **LLaVA**：开创了两阶段训练（对齐 + 微调）和 GPT-4 合成数据的范式（158K 样本）。LLaVA 1.5 进一步引入 AnyRes 动态分辨率、多图像/视频支持、模态自适应数据处理和视觉提示，大幅扩展了 VLM 的能力边界。
3. **Qwen-VL**：采用交叉注意力适配器和 2D 位置嵌入，引入特殊 token 支持边界框输出，并在预训练阶段微调视觉编码器，展现了不同于 LLaVA 的技术路线。讲者曾特别指出，Qwen-VL 将视觉嵌入固定压缩到 256 个 token 的设计“显然不够灵活”，这一限制在后续的 Qwen2-VL 和 Qwen3-VL 中通过动态分辨率和深度嵌入得到了根本性改善。

从 LLaVA 和 Qwen-VL 的初步探索，到后续 Qwen2-VL 和 Qwen3-VL 的系统性演进，VLM 领域在分辨率处理、位置编码、适配器设计和训练策略等方面持续迭代。下一章将深入探讨 Qwen2-VL 和 Qwen3-VL 的具体改进，包括动态分辨率压缩、多模态旋转位置编码（MRoPE）、显式视频时间戳、以及适配器的深度嵌入等前沿技术。

6 Qwen2-VL 与 Qwen3-VL 的演进

第 4 章已经介绍了视觉语言模型（Vision Language Model, VLM）的基础架构范式——冻结视觉编码器、可训练投影器/适配器与语言模型的组合。从 2023 年的 Qwen-VL 出发，阿里巴巴通义千问团队在接下来的两年中进行了系统性的迭代，先后推出 Qwen2-VL 与 Qwen3-VL。本章将聚焦于两代模型的关键改进：动态分辨率、三维位置编码、显式时间戳、长度归一化策略，以及适配器架构的根本性变革。

6.1 Qwen2-VL：动态分辨率与 MRoPE

6.1.1 更大的视觉编码器与动态分辨率

Qwen2-VL 在继承 Qwen-VL 整体框架的基础上，首先升级了视觉编码器（并使用 Chameleon 模型进行初始化），引入了真正意义上的**动态分辨率**（dynamic resolution）机制。回顾第 4 章中 LLaVA 1.5 提出的 AnyRes 方案：将高分辨率图像切分为多个 336×336 的块分别编码后再拼接。Qwen2-VL 的动态分辨率思想与此类似，但在实现上更加精细化。

具体而言，Qwen2-VL 以 224×224 作为基础块尺寸。任意输入图像被切分为若干 224×224 的块，每个块经过 ViT 编码后，再通过压缩操作将每两个 patch 合并为一个 token，最终每个 224×224 的块产生 **66 个视觉 token**。这意味着：

- 一张高分辨率图像可能产生超过 11,000 个视觉 token；
- 一张小尺寸图像（如一个公式截图）可能仅需 8 个 token。

这种自适应的 token 分配方式，使得 Transformer 可以像处理可变长度文本序列一样，灵活地处理不同分辨率的图像，避免了固定尺寸 resize 造成的信息损失或冗余计算。

动

态分辨率的核心思想是：**不对所有图像一视同仁**。Transformer 本身已具备处理可变长度序列的能力，因此图像 token 数量也应当根据实际分辨率自适应调整。

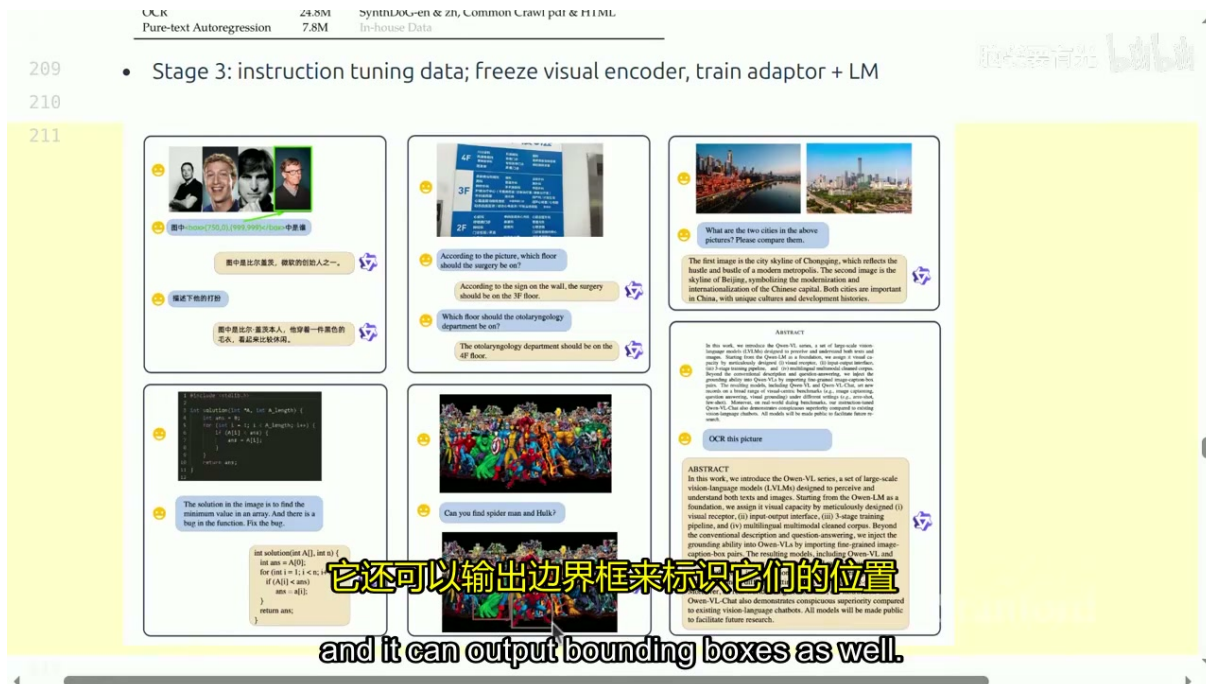


图 10: Qwen2-VL 动态分辨率处理：以 224×224 为基础块，每块经 ViT 编码后压缩为 66 个 token，支持自适应 token 分配

6.1.2 视频处理：2fps 采样与 16K 上限

对于视频输入，Qwen2-VL 采取了务实的折中策略：以**每秒 2 帧**（2fps）的频率进行采样，同时设置**上限为 16,000 个 token**。这意味着对于长视频，模型不会无限制地消耗上下文长度，而是在时间维度上进行稀疏采样。

然而，视频处理引出了一个关键问题：不同样本的 token 数量差异极大。一张单图可能只有几十个 token，而一段长视频可能逼近 16K 上限。如果对所有 token 一视同仁计算交叉熵损失，长视频样本将在梯度更新中占据主导地位，导致模型偏向学习“模糊的视频描述”而非精细的图像理解。

6.1.3 MRoPE：将 RoPE 扩展到三维空间

Qwen2-VL 在位置编码上引入了**多模态旋转位置编码**（Multimodal Rotary Position Embedding, MRoPE）。回顾 RoPE 的核心性质：嵌入向量的内积主要由 token 之间的距离决定，而距离在一维序列中由位置索引之差衡量。

MRoPE 将这一思想扩展到**三维**——高度（height）、宽度（width）和时间（time）：

- 每个图像块或视频帧的位置由一个三元组 (h, w, t) 定义；
- 对三个维度分别独立计算 RoPE，然后将结果**拼接**（concatenate）起来。

$$\text{MRoPE}(h, w, t) = \text{RoPE}_h(h) \oplus \text{RoPE}_w(w) \oplus \text{RoPE}_t(t)$$

其中 $\oplus$ 表示向量拼接操作。这种设计直观且易于实现，使得模型能够区分空间位置与时间顺序。

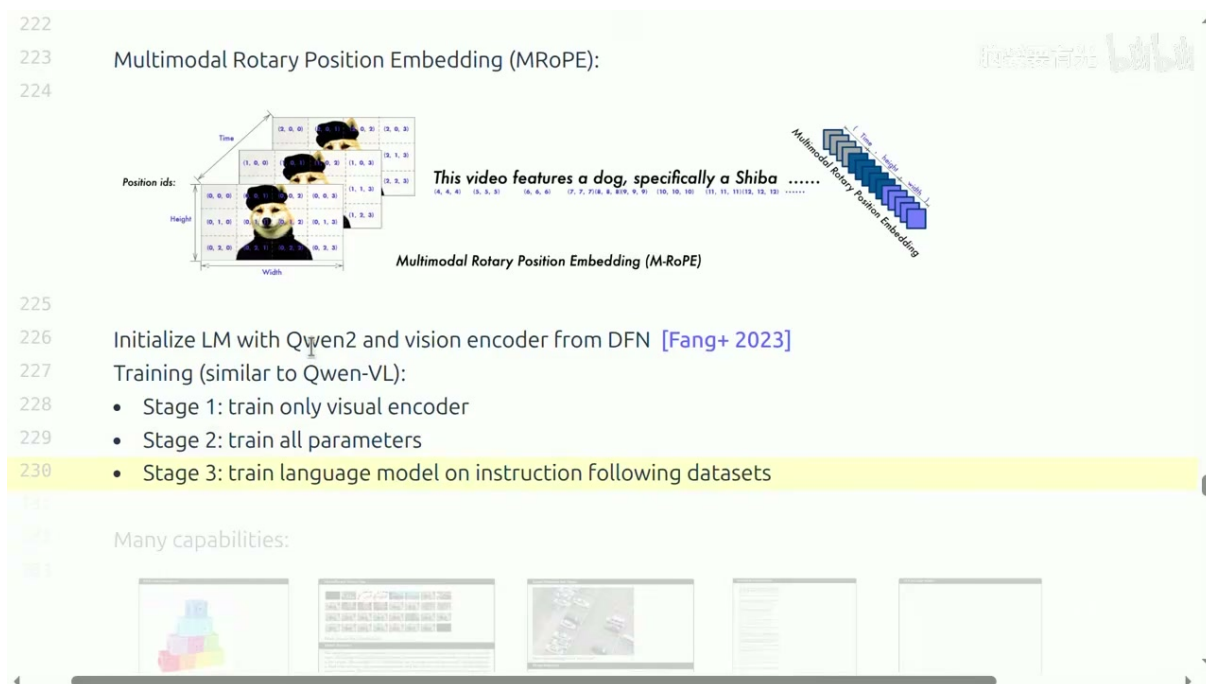


图 11: MRoPE 多模态旋转位置编码：将 RoPE 扩展到三维（高度、宽度、时间），每个维度独立计算后拼接

6.1.4 显式视频时间戳 token

在 Qwen2-VL 之前，视频中的时间信息是**隐式**地编码在位置索引中的——不同帧因为位置不同而获得不同的位置编码，从而间接蕴含了时间先后关系。Qwen2-VL 引入了一种更直接的方式：**显式时间戳 token**。

具体做法是在视频帧序列中插入特殊的 token 来表示绝对时间（如“0 秒”、“2 秒”）。这些时间戳 token 与普通文本 token 一样参与自注意力计算，使得模型可以直接引用时间信息。例如，用户可以提问：“两秒后发生了什么？”模型能够利用显式时间戳 token 定位到对应帧。

显式 vs 隐式时间编码

隐式时间编码：时间信息蕴含在位置索引差异中，模型需要间接推断。

显式时间戳 token：时间信息以可读的 token 形式直接呈现，模型可直接引用。

6.1.5 长度平方根归一化

为解决长视频样本主导损失的问题，Qwen2-VL 采用了**长度平方根归一化**（Square-root Length Normalization）。其核心思想是：不按原始 token 数量加权，而是按 token 数量的平方根进行归一化。

假设一个 batch 中包含不同长度的样本，第 i 个样本有 L_i 个 token，则其损失权重为：

$$w_i = \frac{1}{\sqrt{L_i}}$$

这种归一化有效地“压缩”了长视频样本的影响力。例如，一个包含 10,000 个 token 的视频样本，其权重为 $1/100 = 0.01$ ；而一个 100 个 token 的图像样本，权重为 $1/10 = 0.1$ 。长视频的权重仅比单图高 10 倍，而非 100 倍，从而防止了训练过程中的模态失衡。

长

度归一化与数据混合权重调整是互补的。即使进行了平方根归一化，训练者仍需要根据实际效果进一步调整视频、图像、文本三种模态的采样比例。

6.2 Qwen3-VL：深度嵌入与上下文扩展

6.2.1 基于 Qwen3 的语言模型底座

Qwen3-VL 将语言模型底座升级为 Qwen3 系列——包括密集（dense）模型和 MoE（Mixture of Experts）模型。Qwen3 本身在多语言、代码生成和推理能力上均有显著提升，这为 VLM 的整体质量奠定了坚实基础。

6.2.2 上下文长度的逐步扩展：8K → 32K → 256K

Qwen3-VL 最引人注目的改进之一是上下文长度的极大扩展，最终达到 **256K token**。这对于长视频理解至关重要——一段数小时的视频按 2fps 采样后，很容易产生数万甚至十几万个视觉 token。

Qwen3-VL 采用**渐进式扩展策略**，预训练分为四个阶段：

1. 第一阶段：序列长度 8K，训练适配器；
2. 第二阶段：扩展到 32K；
3. 第三阶段：扩展到 256K；
4. 大部分训练 token 集中在第二和第三阶段。

这种渐进式扩展避免了模型在长序列上直接训练时的不稳定问题，让模型逐步适应越来越长的上下文。

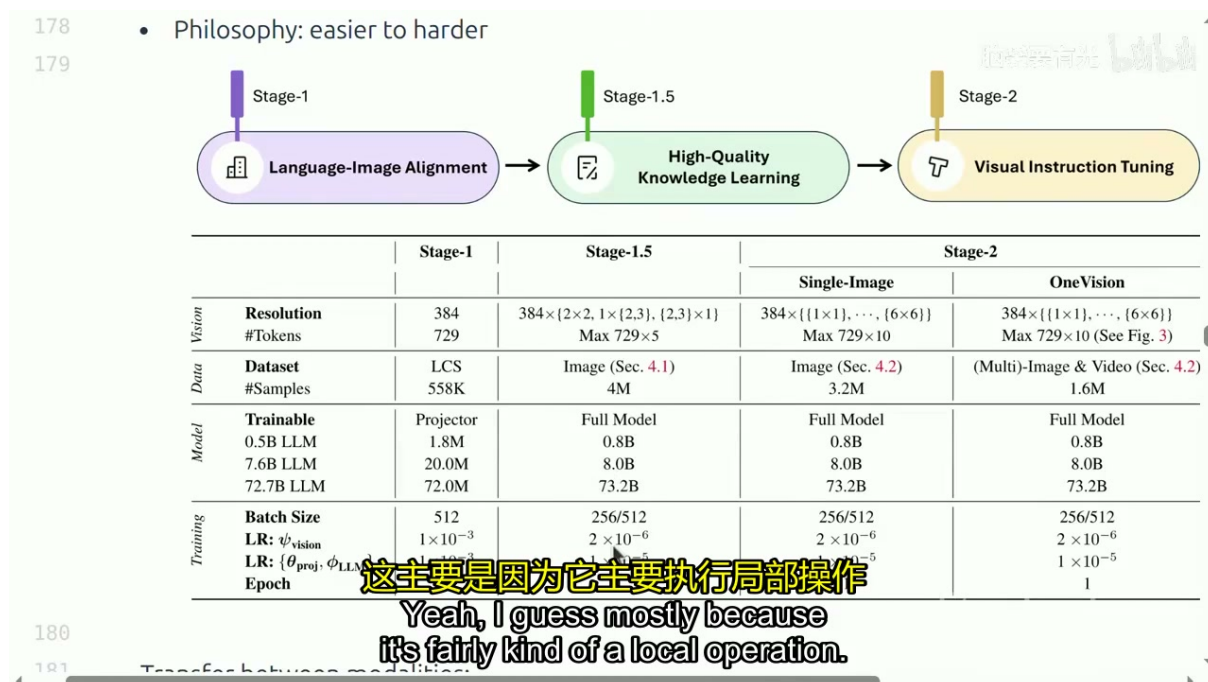


图 12: Qwen3-VL 训练阶段与基准测试表现：多阶段预训练参数配置及在 MMMU、MathVista 等任务上的性能数据

6.2.3 视觉编码器升级：Clip2

Qwen3-VL 的视觉编码器升级为 **Clip2**——CLIP 的一个改进版本。Clip2 的架构与原始 CLIP 完全相同，具有向后兼容性，但经过了重新训练和优化，以更好地服务于 VLM 任务。

6.2.4 3D RoPE 的维度交错改进

Qwen3-VL 对 MRoPE 进行了关键改进。在 Qwen2-VL 中，三个维度的 RoPE 是**顺序拼接**的：先时间维度，再宽度维度，最后高度维度。这导致了一个问题：RoPE 的不同分量代表不同的频率——前面的分量对应低频，后面的分量对应高频。因此，在 Qwen2-VL 中：

- 时间维度占据了所有低频分量；
- 高度维度占据了所有高频分量。

这种频率分配的不均衡是**次优**的。Qwen3-VL 的解决方案是**交错维度**（interleaved dimensions）：将时间、宽度、高度三个维度的频率分量交错排列，使得每个维度都能同时接触到低频和高频信息。

$$\text{Interleaved RoPE} = [\text{RoPE}_t^{(1)}, \text{RoPE}_w^{(1)}, \text{RoPE}_h^{(1)}, \text{RoPE}_t^{(2)}, \text{RoPE}_w^{(2)}, \text{RoPE}_h^{(2)}, \dots]$$

维

度交错的核心洞察：位置编码的频率应当均匀分配给所有空间/时间维度，而非让某一维度独占低频或高频。

6.2.5 深度嵌入适配器：从黑箱投影到残差流注入

Qwen3-VL 在适配器架构上做出了最具创新性的改变。回顾 VLM 适配器的演进路径：

1. LLaVA：最简单的**线性投影**；
2. Qwen-VL：增加**交叉注意力层**；
3. 后续工作：增加 **MLP** 或更复杂的深度堆叠。

Qwen3-VL 提出了一种根本不同的思路——**深度嵌入适配器**（Deeply Embedded Adapter，技术思路源自 DeepSeek 团队）。视觉编码器输出的视觉嵌入不再经过独立的投影模块后作为”外部输入”送入语言模型，而是被**直接注入语言模型各层的残差流**（residual stream）中。

残差流是 Transformer 中逐层传递的隐藏状态 $h^{(l)}$ ，每一层通过自注意力、FFN 等操作对其进行读写，并通过残差连接传递：

$$h^{(l+1)} = h^{(l)} + \text{Layer}^{(l)}(h^{(l)})$$

深度嵌入适配器的做法是：将视觉特征作为额外的”旁路”信息，在语言模型的中间层直接加入残差流。这意味着视觉信息不再仅在输入层与文本拼接，而是在语言模型的深层推理过程中持续参与。

残差流注入 vs 黑箱投影

黑箱投影：视觉编码器 → 投影器 → 固定向量序列 → 语言模型输入层 → 后续各层仅处理文本 + 视觉的混合表示。

深度嵌入：视觉编码器 → 各层残差流直接注入 → 视觉信息在语言模型的每一层都与文本表示交互。

Deepstack, cross-layer fusion to inject visual information into multiple layers [Preprint 2024]

Training:

- Pre-training has 4 stages (train adapter, train all parameters on 8K, 32K, 256K lengths)

Stage	Objective	Training	Token Budget	Sequence Length
S0	Vision-Language Alignment	Merger	67B	8,192
S1	Multimodal Pre-Training	All	~1T	8,192
S2	Long-Context Pre-Training	All	~1T	32,768
S3	Ultra-Long-Context Adaptation	All	100B	262,144

- Post-training: SFT on long CoT data, knowledge distillation, RL

	Benchmark	Qwen3-VL 235B-A22B		Gemini 2.5 Pro		OpenAI GPT-5		Claude Opus 4.1	
		thinking	instruct	thinking	budget-128	high	minimal	thinking	non-thinking
STEM Puzzle	MMMU	80.6	78.7	81.7*	80.9	84.2*	74.4*	78.4	77.2
	MMMU-Pro	69.3	68.1	68.8*	71.2	78.4*	62.7*	64.8	60.7
	MathVista _{mini}	85.8	84.9	82.7*	77.7	81.3	50.9	75.5	74.5
	MathVision	74.6	66.5	73.3*	66.0	70.9	45.8	64.3	57.7
	MathVision _{typ}	63.8	57.0	63.2	56.9	62.8	40.1	54.0	46.4
	We-Math	74.8	67.5	80.6	74.5	73.8	51.8	65.2	60.2
	MathVerse _{mmu}	85.0	72.5	82.9	65.9	84.1	43.0	70.6	68.1
	DynaMath	82.8	79.4	80.0	78.5	85.4	74.0	75.1	72.0
	Math-VR	66.8	65.0	64.7*	54.3	58.1	21.7	54.3	38.0
	ZeroBench	4	2	3	1	2	2	3	1
	VLMscore-BestBbox4	70.5	80.4	86.1	78.5	80.5	53.4	77.8	77.7

图 13: Qwen3-VL 深度嵌入适配器 (DeepStack)：将视觉信息直接注入语言模型各层残差流，实现深层模态融合

6.2.6 后训练：SFT + 知识蒸馏 + RL

Qwen3-VL 的后训练 (post-training) 分为三个阶段：

1. **监督微调 (SFT)**：在高质量多模态指令数据上进行微调；
2. **知识蒸馏 (Knowledge Distillation)**：从更强的模型 (如 GPT-4) 学习；
3. **强化学习 (RL)**：通过奖励模型优化生成质量。

在强化学习阶段，模型可以尝试生成不同的回答，并由奖励模型评判哪个更好——这与预训练阶段对每个 token 进行确定性监督的方式形成对比。

6.2.7 基准表现

Qwen3-VL 在多项基准测试中与闭源模型 (Gemini、GPT-4o、Claude Opus) 展开竞争，在多个任务上达到了同等甚至更好的水平。这表明，通过系统性的架构改进、数据整理和训练策略优化，开源 VLM 已经能够逼近顶尖闭源模型的性能。

6.3 本章小结

Qwen2-VL 和 Qwen3-VL 的演进展示了 VLM 架构在三个维度上的深化：

1. **输入侧精细化**：动态分辨率使图像 token 分配自适应；2fps 采样与 16K 上限平衡了视频质量与上下文消耗；显式时间戳 token 提供了可直接引用的时间信息。
2. **位置编码优化**：从 MRoPE 的三维拼接到 Qwen3-VL 的维度交错，位置编码的频率分配更加均衡。
3. **适配器架构革新**：深度嵌入适配器将视觉信息从”输入层黑箱”解放出来，直接注入语言模型的残差流，实现了更深层次的模态融合。

然而，无论 Qwen2-VL 还是 Qwen3-VL，它们都遵循同一个基本范式：**连续视觉编码器 + 适配器 + 语言模型**。这一范式的根本限制在于——语言模型只能生成文本 token，无法生成图像。接下来的第 6 章将探讨一种试图打破这一限制的激进方案。

7 Chameleon: 全离散化统一多模态

7.1 从连续编码到离散统一

回顾第 4 章和第 5 章讨论的所有 VLM (LLaVA、Qwen-VL、Qwen2-VL、Qwen3-VL)，它们共享同一个架构模式：图像通过连续视觉编码器（如 CLIP ViT）编码为密集向量，再经过适配器投影到语言模型的输入空间。由于语言模型的输出头仅针对文本 token 设计，这些模型只能生成文本，不能生成图像。

解决这一问题有多种路径：

- 在 VLM 之上附加一个**扩散模型头** (diffusion head)，由语言模型输出条件向量来引导图像生成；
- 将图像也转换为**离散 token**，与文本共享同一离散空间，从而用同一个自回归模型统一理解和生成。

2024 年 Meta 提出的 **Chameleon** 选择了第二条路。其核心思想极具美学吸引力：**如果所有模态都是离散 token，那么一个模型就可以用同一种方式理解并生成任何东西。**

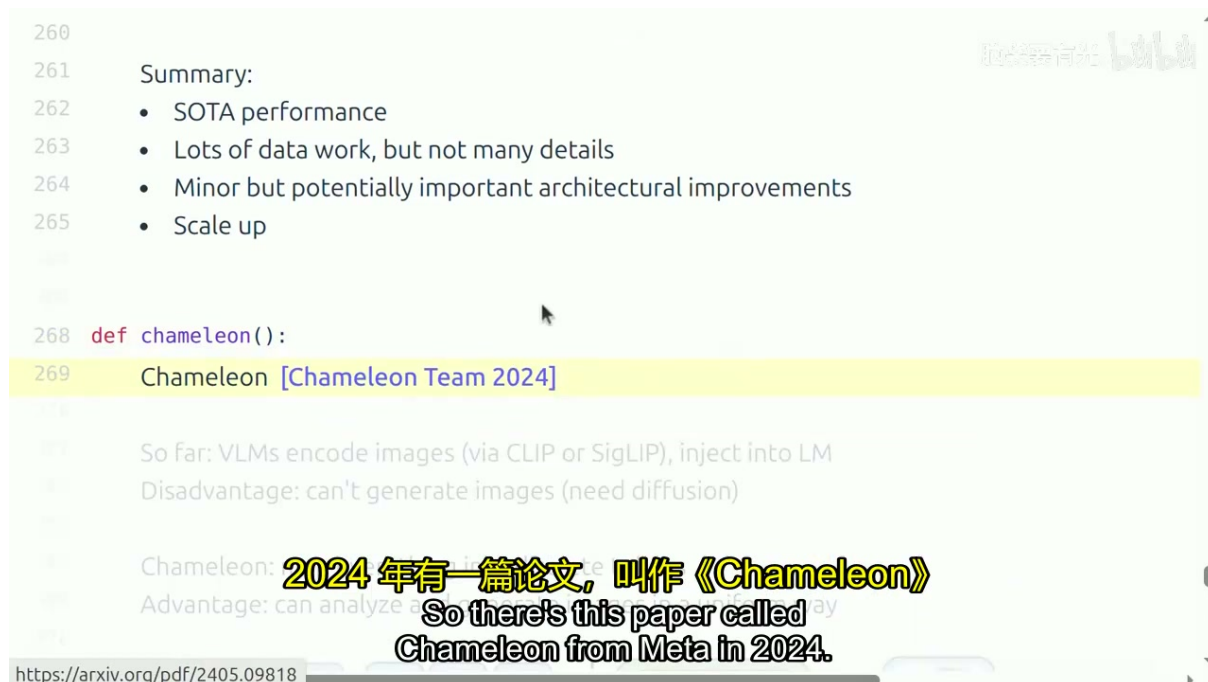


图 14: Chameleon 混合模态自回归语言模型：文本 token 与图像 token 在同一离散空间中交织，统一理解与生成

C

Chameleon 的愿景是全模态 (Omnimodal) 模型的终极简化：文本和图像共享同一个离散空间，输入可以是“文本 + 图像”的任意交错序列，输出也可以是“文本 + 图像”的任意交错序列。

7.2 VQ-VAE: 图像的离散化编码

实现这一愿景的关键技术是**向量量化变分自编码器** (Vector Quantized Variational AutoEncoder, VQ-VAE)。VQ-VAE 的基本流程如下：

1. **编码器 (Encoder)**: 将输入图像映射为连续的潜在向量 z_e ;
2. **量化 (Quantization)**: 在码本 (codebook) 中找到与 z_e 最接近的离散向量 z_q ;
3. **解码器 (Decoder)**: 从 z_q 重建原始图像。

整个系统通过最小化重建误差进行训练。由于量化操作 (argmax/最近邻选择) 不可微, VQ-VAE 通常采用直通估计器 (straight-through estimator) 或额外的 commitment loss 来处理梯度回传。

Chameleon 中 VQ-VAE 的具体参数为:

- 输入图像分辨率: 512×512 ;
- 输出 token 数量: **1024 个**;
- 码本大小 (词汇表): **8192**。

这意味着一张 512×512 的图像被压缩为 1024 个离散 token, 每个 token 从 8192 个可能的”视觉词”中选择。文本和图像 token 共同构成一个扩展的词汇表, Chameleon 为此训练了全新的分词器 (tokenizer) —— 因为混合模态数据的统计特性与普通自然语言文本显著不同, 现有文本分词器无法有效覆盖图像 token 的分布。

$$\text{图像} \xrightarrow{\text{Encoder}} z_e \xrightarrow{\text{Quantize}} z_q \in \{c_1, c_2, \dots, c_{8192}\} \xrightarrow{\text{Decoder}} \hat{\text{图像}}$$

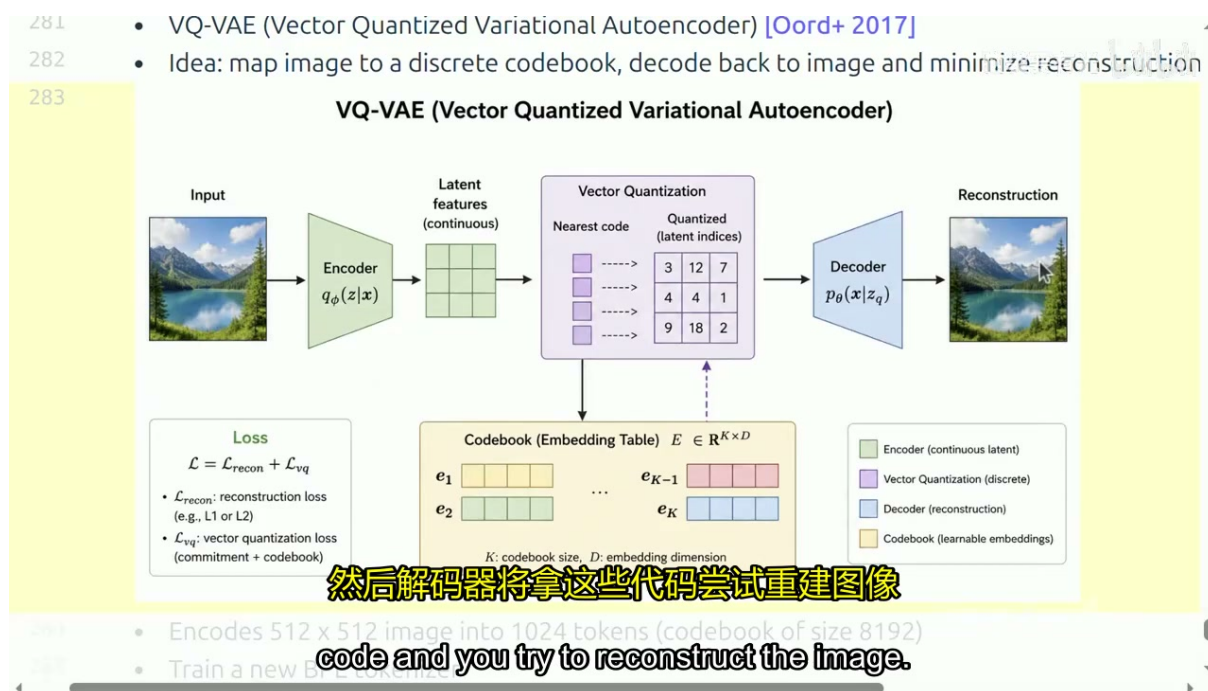


图 15: VQ-VAE 原理: 编码器将图像映射为连续潜在向量, 经码本量化后由解码器重建, 512×512 图像 $\rightarrow$ 1024 个离散 token

7.3 训练: 回归普通语言模型

一旦图像被离散化为 token, Chameleon 的训练就变得异常简洁——它只是普通的语言模型训练:

- 不再需要视觉编码器；
- 不再需要适配器/投影器；
- 不再需要复杂的跨模态对齐损失。

训练数据是文本和图像 token 的交错序列，模型通过标准的下一个 token 预测（next-token prediction）进行学习。Chameleon 的训练分为两个阶段：

1. **大规模预训练**：在海量的图文混合数据上进行监督学习；
2. **高质量微调**：在筛选后的高质量数据上进一步提升生成质量。

Chameleon 的优雅之处

Chameleon 将多模态问题“归约”为单模态问题。一旦图像成为 token，所有关于多模态的复杂性——对齐、融合、跨模态投影——都消失了。剩下的只是训练一个足够大的 Transformer 来预测下一个 token，无论这个 token 来自文本还是图像。

7.4 实际挑战与局限性

尽管概念优雅，Chameleon 在实际中面临多重严峻挑战。

7.4.1 训练不稳定性

文本 token 和图像 token 虽然共享同一离散空间，但它们的统计特性截然不同：

- **文本 token**：预测下一个词时，条件熵相对较低——大多数词语在给定上下文下是可预测的；
- **图像 token**：条件熵非常高——给定周围像素，很难精确预测某个位置的具体颜色值。

这种熵的不均衡导致混合训练时**参数范数增长**和**损失不稳定**。Chameleon 通过两种技术缓解这一问题：

1. **QK 归一化**（QK-normalization）：在注意力计算中对 Query 和 Key 进行归一化，防止内积值过大；
2. **z-loss 正则化**：对 logits 的 log-sum-exp 施加惩罚，控制输出分布的尖锐程度。

$$\mathcal{L}_{z\text{-loss}} = \frac{1}{B} \sum_{i=1}^B \log^2 \sum_{j=1}^V \exp(z_{ij})$$

其中 z_{ij} 是第 i 个样本在第 j 个词汇上的 logit， V 是词汇表大小。z-loss 通过惩罚过大的 logit 值，间接稳定了训练过程中的梯度流。

7.4.2 离散化的信息丢失

VQ-VAE 将连续图像空间压缩到 8192 个离散代码，不可避免地造成**信息丢失**。这一缺陷在某些任务上尤为致命：

- **OCR**：小字体在 $512 \times 512 \rightarrow 1024$ token 的压缩过程中可能变得无法辨认；
- **细粒度细节**：微小的纹理、边缘信息在量化过程中被平滑或丢失。

离

散化是 Chameleon 的阿喀琉斯之踵。对于需要精确像素级理解的任务（如 OCR、小文字阅读），VQ-VAE 的压缩率 ($512 \times 512 \times 3 \approx 786\text{K bits}$ 压缩到 $1024 \times \log_2 8192 \approx 13\text{K bits}$ ，压缩比约 60:1) 过高，导致关键信息不可逆地丢失。

7.4.3 多模态联合训练的困难

即使解决了训练稳定性问题，文本和图像 token 的本质差异也意味着联合训练需要极其精细的**数据混合和权重调整**。这与第 5 章中讨论的视频-文本权重问题类似，但在 Chameleon 中更为尖锐——因为图像 token 的预测难度远高于文本 token。

7.5 历史地位：从流行到式微

VQ-VAE 的概念最早可追溯至 2017 年的原始论文，及其后续变体（如 VQ-GAN、DALL-E）在 2019-2021 年间曾是图像生成领域的主流方法。当时，Transformer 只能生成离散 token，因此将图像离散化是“自然”的选择。

然而，随着**扩散模型**（diffusion model）的兴起，连续空间的图像生成取得了突破性进展。扩散模型通过在像素空间或潜在空间中逐步去噪，能够生成极高质量的图像，且避免了离散化的信息瓶颈。因此，全离散化统一方案（如 Chameleon）的热度逐渐下降。

离散化 vs 扩散模型

离散化方案（VQ-VAE + Transformer）的优势在于统一性——一个模型处理所有模态。

扩散模型的优势在于生成质量——在像素/连续潜在空间中保留完整的细粒度信息。

当前前沿实践倾向于：**连续编码器理解 + 扩散模型生成**的分离架构。

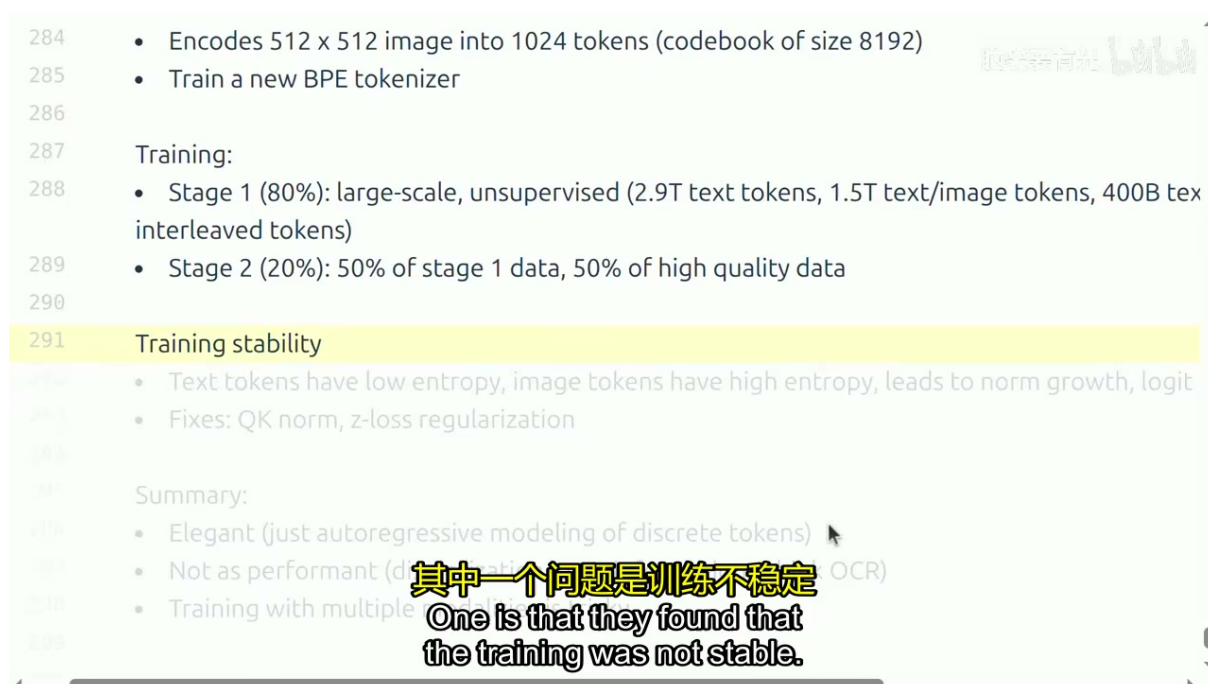


图 16: Chameleon 训练稳定性挑战：文本 token 低熵与图像 token 高熵的差异导致参数范数增长，需 QK 归一化与 z-loss 缓解

7.6 本章小结

Chameleon 代表了一种激进的架构理想：将所有模态统一为离散 token，用最简洁的自回归模型同时实现理解与生成。其核心贡献和教训可以总结为：

1. **VQ-VAE 离散化**：512 × 512 图像 → 1024 个 token，码本大小 8192，为图文共享空间奠定基础；
2. **训练简洁性**：无需适配器、无需视觉编码器，普通语言模型训练即可；
3. **训练不稳定性**：文本低熵与图像高熵的冲突导致参数范数增长，需 QK 归一化和 z-loss 缓解；
4. **信息丢失**：高压缩比的离散化在 OCR 等细粒度任务上表现不佳；
5. **历史趋势**：扩散模型的兴起使得全离散化方案逐渐让位于”连续理解 + 扩散生成”的混合架构。

Chameleon 与 Qwen3-VL 形成了有趣的对比：Qwen3-VL 将视觉信息深度嵌入语言模型的每一层残差流，是”连续编码器 + 深度融合”的极端；Chameleon 则完全抛弃连续编码器，追求”完全离散、完全统一”的极端。当前业界的主流实践位于这两者之间——使用连续编码器（如 CLIP）进行理解，使用扩散模型进行生成。